

Hình 11-1. Độ bão hòa của hàm kích hoạt sigmoid

Khởi tạo Glorot và He

Trong bài báo của mình, Glorot và Bengio đề xuất một cách để giảm thiểu đáng kể vấn đề độ dốc không ổn định. Họ chỉ ra rằng chúng ta cần tín hiệu truyền đúng cách theo cả hai hướng: theo hướng thuận khi đưa ra dự đoán và theo hướng ngược lại khi lan truyền ngược độ dốc. Chúng ta không muốn tín hiệu bị tắt hẳn, cũng không muốn nó bùng nổ và bão hòa. Để tín hiệu truyền tải đúng cách, các tác giả lập luận rằng chúng ta cần phương sai của đầu ra của mỗi lớp bằng với phương sai của đầu vào của nó, và chúng ta cần độ dốc có phương sai bằng nhau trước và sau khi truyền qua một lớp theo hướng ngược lại (vui lòng xem bài báo nếu bạn quan tâm đến các chi tiết toán học). Trên thực tế, không thể đảm bảo cả hai trừ khi lớp đó có số lượng đầu vào và đầu ra bằng nhau (các số này được gọi là fan-in và fan-out của lớp), nhưng Glorot và Bengio đã đề xuất một sự thỏa hiệp tốt đã được chứng minh là hoạt động rất hiệu quả trong thực tế: trọng số kết nối của mỗi lớp phải được khởi tạo ngẫu nhiên như mô tả trong **Phương trình 11-1**, trong đó $\text{fan} = (\text{fan}_{\text{in}} + \text{fan}_{\text{out}}) / 2$. Chiến lược khởi tạo này được gọi là khởi tạo Xavier hoặc khởi tạo Glorot, theo tên tác giả đầu tiên của bài báo.

Phương trình 11-1. Khởi tạo Glorot (khi sử dụng hàm kích hoạt sigmoid)

Phân phối chuẩn với giá trị trung bình bằng 0 và phương sai $\sigma^2 = \frac{1}{\text{fan}_{\text{avg}}}$

Hoặc phân bố đều giữa $-r$ và $+r$, với $r = \sqrt{3 / \text{fan}_{\text{avg}}}$

Nếu bạn thay thế fan bằng fan trong **Phương trình 11-1**, bạn sẽ nhận được một chiến lược khởi tạo mà Yann LeCun đã đề xuất vào những năm 1990. Ông gọi đó là phương pháp khởi tạo LeCun. Genevieve Orr và Klaus-Robert Müller thậm chí còn đề xuất nó trong cuốn sách Neural Networks: Tricks of the Trade (Springer) xuất bản năm 1998 của họ. Khởi tạo LeCun tương đương với khởi tạo Glorot khi $\text{fan}_{\text{in}} = \text{fan}_{\text{out}}$.

ngoài Phải mất hơn một thập kỷ các nhà nghiên cứu mới nhận ra tầm quan trọng của điều này.

Mấu chốt là: Sử dụng phương pháp khởi tạo Glorot có thể tăng tốc quá trình huấn luyện đáng kể, và đây là một trong những thủ thuật dẫn đến sự thành công của Học sâu (Deep Learning).

Một số bài báo đã đưa ra các chiến lược tương tự cho các hàm kích hoạt khác nhau. Các chiến lược này chỉ khác nhau ở quy mô của phương sai và việc chúng sử dụng fan hay fan³, như được hiển thị trong **Bảng 11-1** (đối với trung bình đồng nhất)

phân phối, chỉ cần sử dụng $\sigma^2 = \frac{2}{n + m}$). **Chiến lược khởi tạo** Đối với hàm kích hoạt ReLU và các biến thể của nó, phương pháp khởi tạo được gọi là khởi tạo He hoặc khởi tạo Kaiming, theo tên tác giả đầu tiên của bài báo. Đối với SELU, hãy sử dụng phương pháp khởi tạo của Yann LeCun, tốt nhất là với phân phối Chuẩn. Chúng ta sẽ đề cập đến tất cả các hàm kích hoạt này trong thời gian ngắn tới.

Tab

1e11-1

•

TRONG

itia

Liza

than số

và

esthère

c
h

ty p
e
o

f

Một

ct
iv

Một

ti
o

N

f

bạn

N
ct

Tu

o
N

Khởi tạo	Hàm kích hoạt σ^2 (Bình thường)
----------	--

Xavier Glorot	Không có, tanh, sigmoid, softmax	1 / người hâm mộ trung bình
Khai Minh Hà	ReLU, Leaky ReLU, ELU, GELU, Swish, Mish 2 / quạt	TRONG
Yann LeCun	SELU	1 / người hâm mộ

Theo mặc định, Keras sử dụng phương pháp khởi tạo Glorot với phân bố đồng đều. Khi tạo một lớp, bạn có thể thay đổi điều này thành khởi tạo He bằng cách đặt `kernel_initializer="he_uniform"` hoặc `kernel_initializer="he_normal"` như sau:

```
import tensorflow as tf
```

```
dense = tf.keras.layers.Dense(50, activation="relu",  
                                kernel_initializer="he_normal")
```

Ngoài ra, bạn cũng có thể lấy bất kỳ giá trị khởi tạo nào được liệt kê trong **Bảng 11-1** và nhiều hơn nữa bằng cách sử dụng trình khởi tạo VarianceScaling. Ví dụ, nếu bạn muốn khởi tạo He với phân phối đồng đều và dựa trên fan trung bình (thay vì fan trung bình), bạn có thể sử dụng đoạn mã sau:

```
he_avg_init = tf.keras.initializers.VarianceScaling(scale=2., mode="fan_avg",  
distribution="uniform")  
dense = tf.keras.layers.Dense(50, activation="sigmoid",  
                                kernel_initializer=he_avg_init)
```

Chức năng kích hoạt tốt hơn

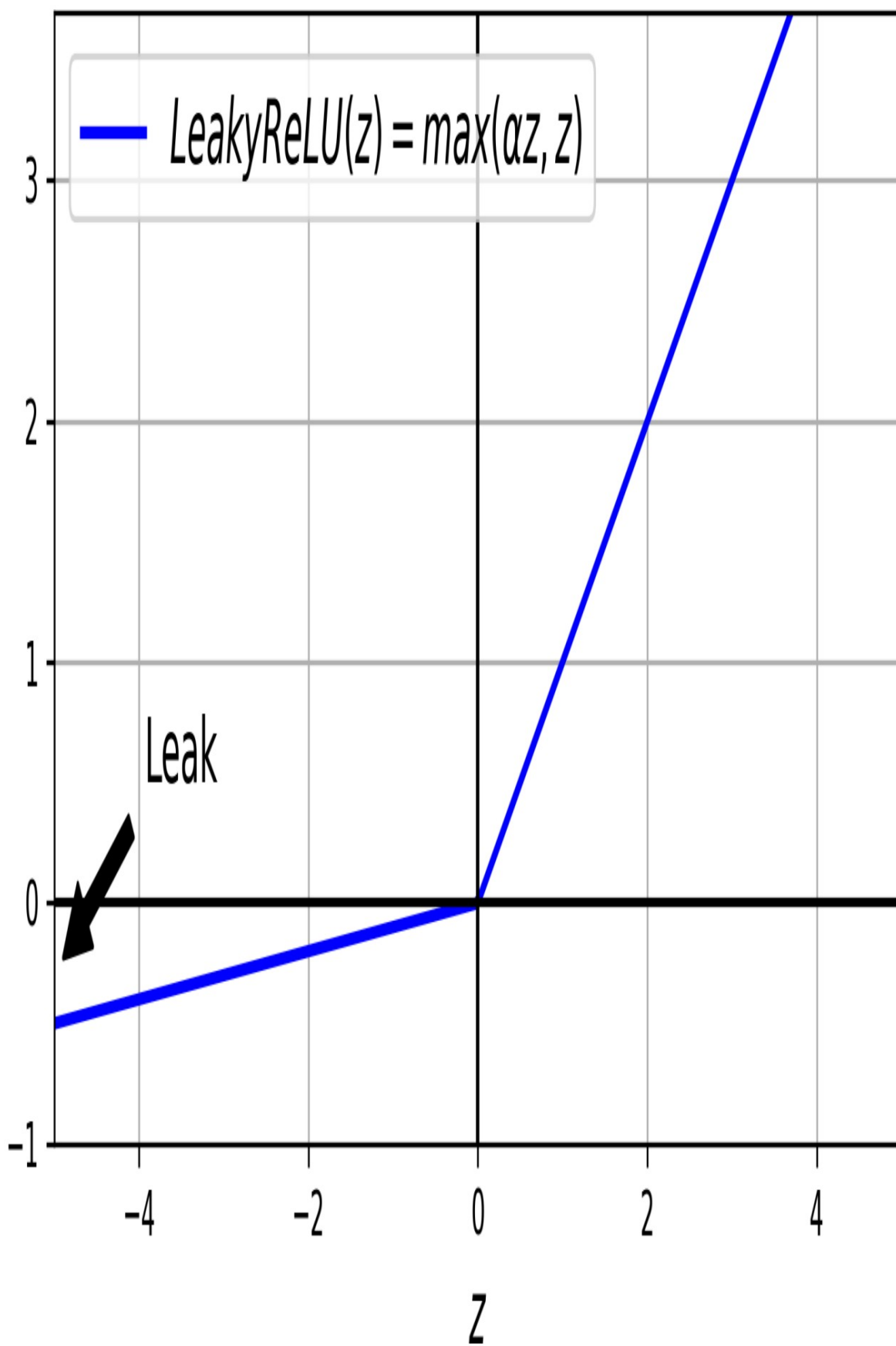
Một trong những phát hiện quan trọng trong bài báo năm 2010 của Glorot và Bengio là các vấn đề về độ dốc không ổn định một phần là do lựa chọn hàm kích hoạt không phù hợp. Cho đến lúc đó, hầu hết mọi người đều cho rằng nếu tạo hóa đã chọn sử dụng các hàm kích hoạt gần giống sigmoid trong các tế bào thần kinh sinh học, thì đó hẳn là một lựa chọn tuyệt vời. Nhưng hóa ra các hàm kích hoạt khác hoạt động tốt hơn nhiều trong mạng nơ-ron sâu—đặc biệt là hàm kích hoạt ReLU, chủ yếu vì nó không bị bão hòa đối với các giá trị dương, và cũng vì nó tính toán rất nhanh.

Thật không may, hàm kích hoạt ReLU không hoàn hảo. Nó gặp phải vấn đề được gọi là "ReLU chết": trong quá trình huấn luyện, một số nơ-ron thực sự "chết", nghĩa là chúng ngừng xuất ra bất cứ thứ gì khác ngoài 0. Trong một số trường hợp, bạn có thể thấy rằng một nửa số nơ-ron trong mạng của bạn đã chết, đặc biệt nếu bạn sử dụng tốc độ học lớn. Một nơ-ron chết khi trọng số của nó được điều chỉnh theo cách mà đầu vào của hàm ReLU (tức là tổng trọng số của đầu vào của nơ-ron cộng với số hạng thiên vị của nó) là âm đối với tất cả các trường hợp trong tập huấn luyện. Khi điều này xảy ra, nó chỉ tiếp tục xuất ra số 0, và thuật toán Gradient Descent không còn tác động đến nó nữa vì gradient Hệ số của hàm ReLU bằng 0 khi đầu vào là số âm.

Để giải quyết vấn đề này, bạn có thể muốn sử dụng một biến thể của hàm ReLU, chẳng hạn như leaky ReLU.

ReLU rò rỉ

Hàm kích hoạt Leaky ReLU được định nghĩa là $\text{LeakyReLU}(z) = \max(\alpha z, z)$ (xem [Hình 11-2](#)). Siêu tham số α xác định mức độ "rò rỉ" của hàm: đó là độ dốc của hàm đối với $z < 0$. Việc có độ dốc đối với $z < 0$ đảm bảo rằng các hàm Leaky ReLU không bao giờ chết; chúng có thể rơi vào trạng thái hôn mê dài, nhưng 5
Họ có cơ hội tỉnh giấc. Một [bài báo năm 2015](#) Bài báo của Bing Xu và cộng sự đã so sánh một số biến thể của hàm kích hoạt ReLU, và một trong những kết luận của họ là các biến thể "rò rỉ" (leaky) luôn cho hiệu suất tốt hơn hàm kích hoạt ReLU nghiêm ngặt (strict ReLU). Trên thực tế, việc đặt $\alpha = 0.2$ (rò rỉ lớn) dường như mang lại hiệu suất tốt hơn so với $\alpha = 0.01$ (rò rỉ nhỏ). Bài báo cũng đánh giá hàm ReLU rò rỉ ngẫu nhiên (RReLU), trong đó α được chọn ngẫu nhiên trong một phạm vi nhất định trong quá trình huấn luyện và được cố định ở giá trị trung bình trong quá trình kiểm tra. RReLU cũng hoạt động khá tốt và dường như đóng vai trò như một bộ điều chỉnh, giảm nguy cơ quá khớp tập huấn luyện. Cuối cùng, bài báo đã đánh giá hàm ReLU rò rỉ tham số (PReLU), trong đó α được phép học trong quá trình huấn luyện: thay vì là một siêu tham số, nó trở thành một tham số có thể được sửa đổi bằng lan truyền ngược giống như bất kỳ tham số nào khác. PReLU được báo cáo là vượt trội hơn hẳn ReLU trên các tập dữ liệu hình ảnh lớn, nhưng trên các tập dữ liệu nhỏ hơn, nó có nguy cơ quá khớp tập huấn luyện.



Hình 11-2. Leaky ReLU: tương tự như ReLU, nhưng có độ dốc nhỏ đối với các giá trị âm.

Keras bao gồm các lớp LeakyReLU và PReLU trong gói

`tf.keras.layers`. Giống như các biến thể ReLU khác, bạn nên sử dụng khởi tạo He, ví dụ:

```
leaky_relu = tf.keras.layers.LeakyReLU(alpha=0.2) # mặc định là alpha=0.3
dense = tf.keras.layers.Dense(50, activation=leaky_relu,
                              kernel_initializer="he_normal")
```

Nếu muốn, bạn cũng có thể sử dụng LeakyReLU như một lớp riêng biệt trong mô hình của mình, điều này không tạo ra sự khác biệt nào đối với quá trình huấn luyện và dự đoán:

```
model = tf.keras.models.Sequential([ [...] #
    nhiều lớp hơn
    tf.keras.layers.Dense(50, kernel_initializer="he_normal"), #
    không kích hoạt
    tf.keras.layers.LeakyReLU(alpha=0.2), # kích hoạt như một lớp riêng biệt
    [...] # thêm các
    lớp
])
```

Đối với PReLU, hãy thay thế LeakyReLU bằng PReLU. Hiện tại Keras chưa có triển khai chính thức nào của RReLU, nhưng bạn có thể dễ dàng tự triển khai (để tìm hiểu cách làm, hãy xem các bài tập ở cuối [Chương 12](#)).

ReLU, leaky ReLU và PReLU đều gặp phải vấn đề là chúng không phải là các hàm trơn: đạo hàm của chúng thay đổi đột ngột (tại $z = 0$). Như chúng ta đã thấy trong [Chương 4](#) khi thảo luận về Lasso, loại gián đoạn này có thể khiến thuật toán Gradient Descent dao động quanh điểm tối ưu và làm chậm quá trình hội tụ. Vì vậy, bây giờ chúng ta sẽ xem xét các biến thể trơn của hàm kích hoạt ReLU, bắt đầu với ELU và SELU.

ELU và SELU

Trong một [bài báo năm 2016](#) ⁶ Một phương pháp được đề xuất bởi ⁶, một chức năng kích hoạt mới Djork-Arne Clevert và cộng sự, được gọi là đơn vị tuyến tính hàm mũ (ELU), đã cho hiệu quả vượt trội hơn.

Tất cả các biến thể ReLU trong các thí nghiệm của các tác giả đều cho thấy: thời gian huấn luyện được rút ngắn và mạng nơ-ron hoạt động tốt hơn trên tập dữ liệu kiểm tra. Phương trình 11-2 thể hiện định nghĩa của hàm kích hoạt này.

Phương trình 11-2. Hàm kích hoạt ELU

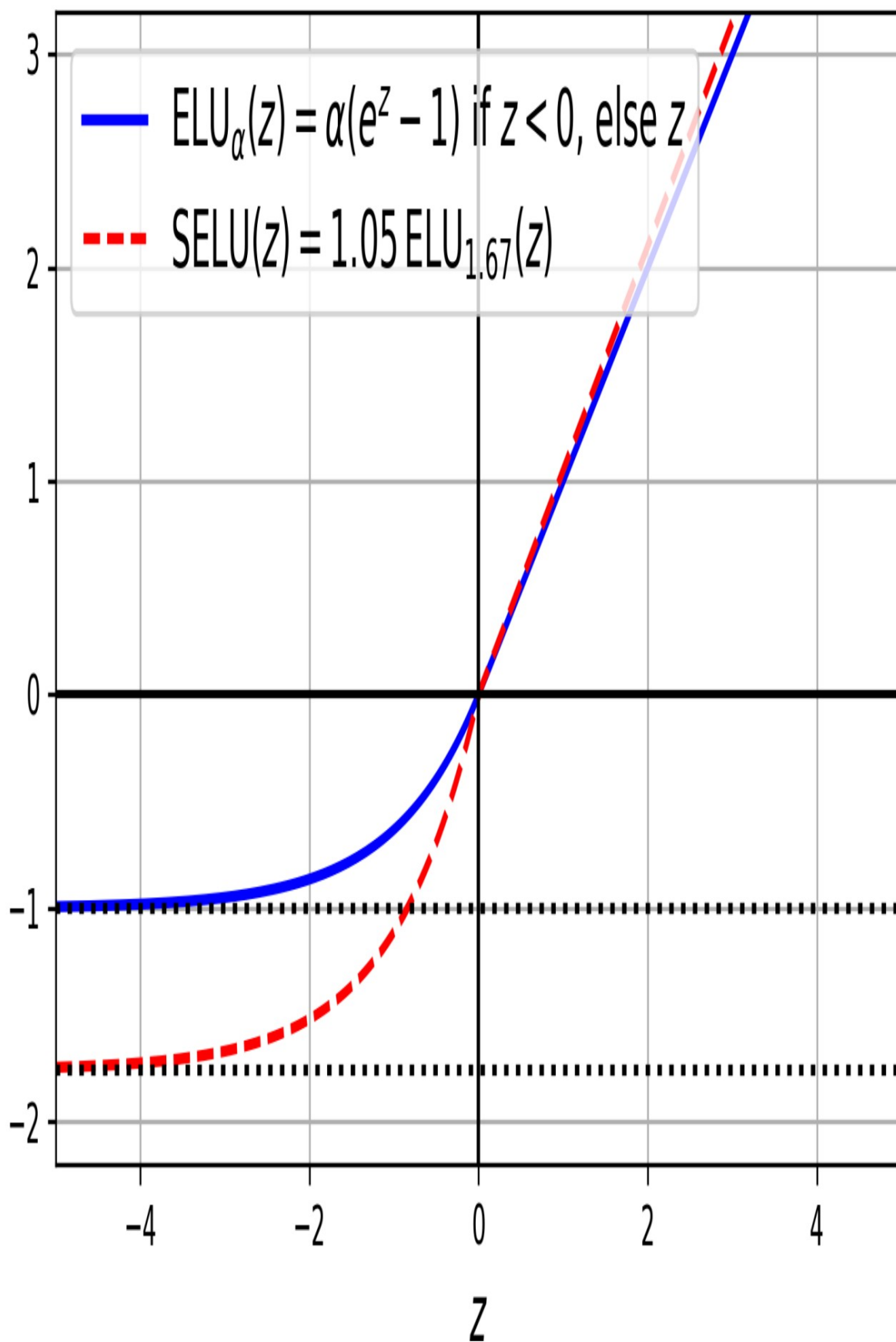
$$\text{ELU}\alpha(z) = \begin{cases} \alpha(\exp(z)) & \text{nếu } z < 0 \\ z & \text{nếu } z \geq 0 \end{cases}$$

Hàm kích hoạt ELU trông rất giống hàm ReLU (xem

Hình 11-3), với một vài điểm khác biệt chính:

- Nó nhận giá trị âm khi $z < 0$, điều này cho phép đơn vị có đầu ra trung bình gần bằng 0 hơn và giúp giảm thiểu vấn đề suy giảm độ dốc. Siêu tham số α xác định giá trị ngược lại với giá trị mà hàm ELU tiến tới khi z là một số âm lớn. Nó thường được đặt là 1, nhưng bạn có thể điều chỉnh nó giống như bất kỳ siêu tham số nào khác.
- Nó có độ dốc khác 0 đối với $z < 0$, điều này giúp tránh được vấn đề các nơ-ron chết.
- Nếu α bằng 1 thì hàm số sẽ trơn tru ở mọi nơi, kể cả xung quanh $z = 0$, điều này giúp tăng tốc độ của thuật toán Gradient Descent vì nó không bị dao động nhiều sang trái và phải quanh $z = 0$.

Việc sử dụng ELU với Keras rất dễ dàng, chỉ cần thiết lập `activation="elu"`, và giống như các biến thể ReLU khác, bạn nên sử dụng khởi tạo He. Nhược điểm chính của hàm kích hoạt ELU là tốc độ tính toán chậm hơn so với hàm ReLU và các biến thể của nó (do sử dụng hàm mũ). Tốc độ hội tụ nhanh hơn trong quá trình huấn luyện có thể bù đắp cho tốc độ tính toán chậm đó, nhưng dù vậy, khi kiểm thử, mạng ELU sẽ chậm hơn một chút so với mạng ReLU.



Hình 11-3. Chức năng kích hoạt ELU và SELU

Một năm sau ELU, một [bài báo năm 2017](#) ⁷ Günter Klambauer và cộng sự đã giới thiệu hàm kích hoạt Scaled ELU (SELU): như tên gọi của nó, đây là một biến thể được điều chỉnh tỷ lệ của hàm kích hoạt ELU (khoảng 1,05 lần ELU, sử dụng $\alpha \approx 1,67$). Các tác giả đã chỉ ra rằng nếu bạn xây dựng một mạng nơ-ron chỉ bao gồm một chồng các lớp dày đặc (tức là một MLP), và nếu tất cả các lớp ẩn đều sử dụng hàm kích hoạt SELU, thì mạng sẽ tự chuẩn hóa: đầu ra của mỗi lớp sẽ có xu hướng duy trì giá trị trung bình là 0 và độ lệch chuẩn là 1 trong quá trình huấn luyện, điều này giải quyết được vấn đề gradient biến mất/bùng nổ. Kết quả là, hàm kích hoạt SELU có thể hoạt động tốt hơn các hàm kích hoạt khác đối với MLP, đặc biệt là các MLP sâu. Để sử dụng nó với Keras, chỉ cần đặt `activation="selu"`. Tuy nhiên, có một vài điều kiện để quá trình tự chuẩn hóa xảy ra (xem bài báo để biết lý do toán học):

- Các đặc trưng đầu vào phải được chuẩn hóa: giá trị trung bình bằng 0 và độ lệch chuẩn bằng 1.
- Trọng số của mỗi lớp ẩn phải được khởi tạo bằng phương pháp khởi tạo chuẩn LeCun. Trong Keras, điều này có nghĩa là thiết lập `kernel_initializer="lecun_normal"`.
- Tính chất tự chuẩn hóa chỉ được đảm bảo với các mạng MLP thông thường. Nếu bạn thử sử dụng SELU trong các kiến trúc khác, chẳng hạn như mạng nơ-ron hồi quy (xem [Chương 15](#)) hoặc mạng có kết nối bỏ qua (tức là các kết nối bỏ qua các lớp, như trong mạng Wide & Deep), thì có lẽ nó sẽ không vượt trội hơn ELU.
- Bạn không thể sử dụng các kỹ thuật điều chỉnh như điều chỉnh l_1 hoặc l_2 , chuẩn tối đa, chuẩn theo lô hoặc dropout thông thường (những kỹ thuật này sẽ được thảo luận sau trong chương này).

Đây là những hạn chế đáng kể, vì vậy, bất chấp những lời hứa hẹn, SELU đã không thu hút được nhiều sự chú ý. Hơn nữa, ba hàm kích hoạt khác dường như hoạt động tốt hơn nó một cách khá nhất quán trong hầu hết các nhiệm vụ: GELU, Swish và Mish.

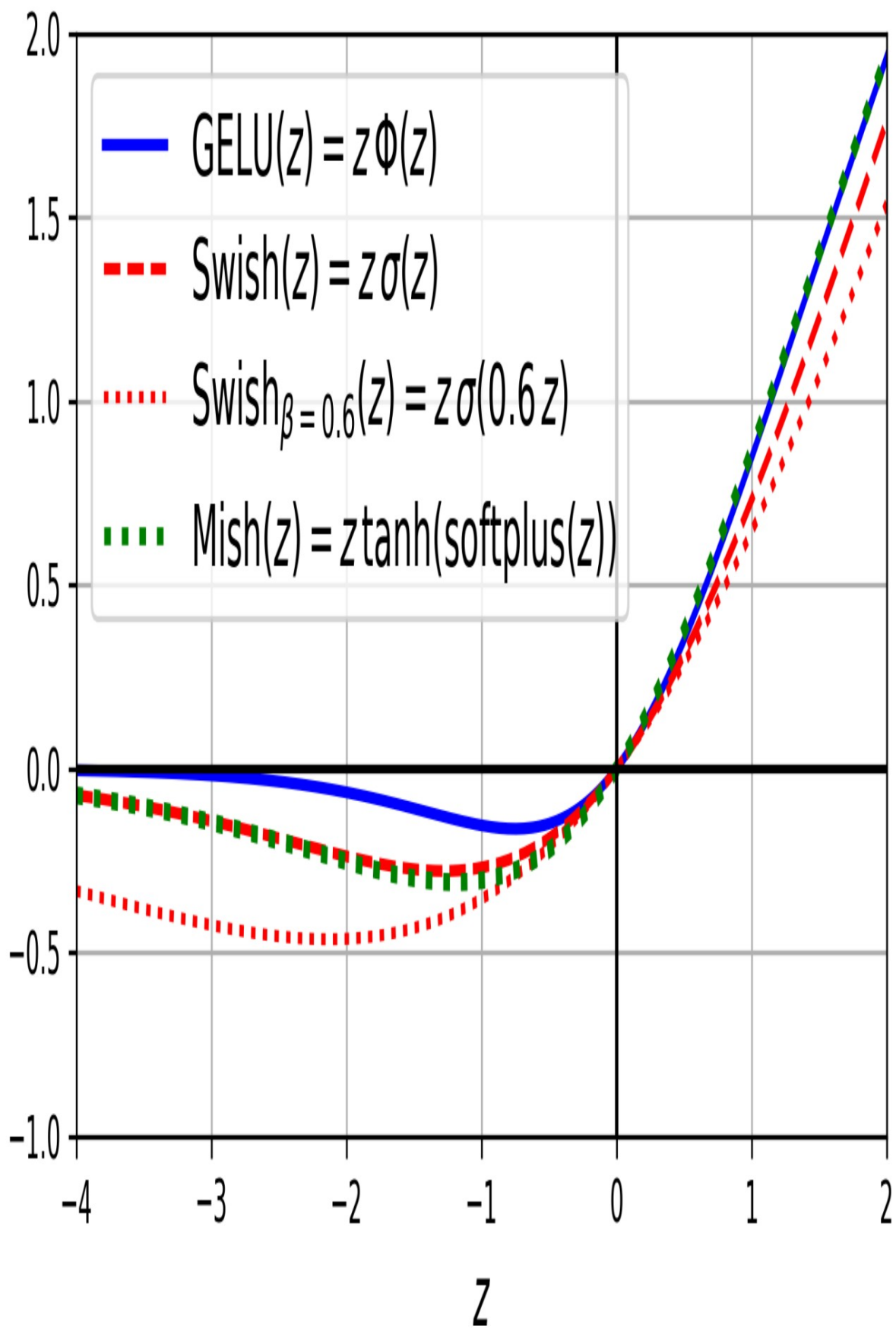
GELU, Swish và Mish

GELU được giới thiệu trong một [bài báo năm 2016](#), bởi Dan Hendrycks và Kevin Gimpel. Một lần nữa⁸, bạn có thể coi nó như một biến thể mượt mà của hàm kích hoạt ReLU. Định nghĩa của nó được đưa ra trong [Phương trình 11-3](#), trong đó Φ là hàm phân phối tích lũy Gaussian chuẩn (CDF): $\Phi(z)$ tương ứng với xác suất mà một giá trị được lấy mẫu ngẫu nhiên từ phân phối Chuẩn có trung bình 0 và phương sai 1 thấp hơn z .

Phương trình 11-3. Hàm kích hoạt GELU

$$\text{GELU}(z) = z \Phi(z)$$

Như bạn có thể thấy trong [Hình 11-4](#), GELU tương tự như ReLU: nó tiến đến 0 khi đầu vào z rất âm và tiến đến z khi z rất dương. Tuy nhiên, trong khi tất cả các hàm kích hoạt mà chúng ta đã thảo luận cho đến nay đều lồi và đơn điệu, thì hàm kích hoạt GELU lại không như vậy: từ trái sang phải⁹, nó bắt đầu bằng một đường thẳng, sau đó uốn lượn xuống, đạt đến điểm thấp khoảng -0.17 (gần $z \approx -0.75$), và cuối cùng nó bật lên và kết thúc bằng một đường thẳng hướng lên trên bên phải. Hình dạng khá phức tạp này và thực tế là nó có độ cong ở mọi điểm có thể giải thích tại sao nó hoạt động tốt như vậy, đặc biệt là đối với các tác vụ phức tạp: Thuật toán Gradient Descent có thể dễ dàng hơn trong việc khớp các mẫu phức tạp. Trên thực tế, nó thường vượt trội hơn mọi hàm kích hoạt khác đã được thảo luận cho đến nay. Tuy nhiên, nó tốn nhiều tài nguyên tính toán hơn một chút, và sự tăng hiệu suất mà nó mang lại không phải lúc nào cũng đủ để biện minh cho chi phí bổ sung. Tuy nhiên, có thể chứng minh rằng nó xấp xỉ bằng $\sigma(1.702 z)$, trong đó σ là hàm sigmoid: sử dụng phép xấp xỉ này cũng hoạt động rất tốt và có ưu điểm là tính toán nhanh hơn nhiều. Trên thực tế, bài báo về GELU cũng giới thiệu hàm kích hoạt SiLU, bằng $\sigma(z)$, nhưng nó đã bị GELU vượt trội hơn trong các thử nghiệm của họ.



Hình 11-4. Các hàm kích hoạt GELU, Swish, Swish tham số hóa và Mish.

Điều thú vị là, một bài báo năm 2017 bởi Prajit Ramachandran và cộng sự. khám phá lại Chức năng của SiLU là tự động tìm kiếm các hàm kích hoạt tốt. ¹⁰

Họ đặt tên cho nó là Swish, và cái tên này nhanh chóng được chấp nhận. Trong bài báo của họ, Swish cho thấy hiệu quả vượt trội hơn tất cả các hàm khác, bao gồm cả GELU. Sau đó, các tác giả đã tổng quát hóa Swish bằng cách thêm một siêu tham số β để điều chỉnh đầu vào của hàm sigmoid. Hàm Swish tổng quát là $\text{Swish}(z) = z\sigma(\beta z)$, vì vậy GELU xấp xỉ bằng hàm Swish tổng quát khi sử dụng $\beta = 1,702$. Bạn có thể điều chỉnh β giống như bất kỳ siêu tham số nào khác. Ngoài ra, cũng có thể làm cho β có thể huấn luyện được và để thuật toán Gradient Descent tối ưu hóa nó: tương tự như PReLU, điều này có thể làm cho mô hình của bạn mạnh mẽ hơn, nhưng cũng có nguy cơ bị quá khớp dữ liệu.

Một hàm kích hoạt khá tương tự khác là Mish, được giới thiệu trong một bài báo năm 2019. Do Diganta Misra đề xuất. Nó được định nghĩa là $\text{mish}(z) = z \tanh(\text{softplus}(z))$, trong đó $\text{softplus}(z) = \log(1 + \exp(z))$. ¹¹ Giống như GELU và Swish, nó là một biến thể trơn tru, không lỗi và không đơn điệu của ReLU, và một lần nữa tác giả đã thực hiện nhiều thí nghiệm và nhận thấy rằng Mish nhìn chung hoạt động tốt hơn các hàm kích hoạt khác, thậm chí cả Swish và GELU với một biên độ nhỏ. Hình 11-4 hiển thị GELU, Swish (cả với β mặc định = 1 và với $\beta = 0,6$), và cuối cùng là Mish. Như bạn có thể thấy, Mish gần như trùng khớp hoàn hảo với Swish khi z âm, và gần như trùng khớp hoàn hảo với GELU khi z dương.

Keras hỗ trợ GELU và Swish ngay từ đầu, chỉ cần sử dụng `activation="gelu"` hoặc `activation="swish"`. Tuy nhiên, nó chưa hỗ trợ Mish hoặc hàm kích hoạt Swish tổng quát (nhưng hãy xem Chương 12 để biết cách triển khai các hàm kích hoạt và lớp của riêng bạn).

MẸO

Vậy, bạn nên sử dụng hàm kích hoạt nào cho các lớp ẩn của mạng nơ-ron sâu? ReLU vẫn là lựa chọn mặc định tốt cho các tác vụ đơn giản: nó thường tốt tương đương với các hàm kích hoạt phức tạp hơn, cộng thêm tốc độ tính toán rất nhanh, và nhiều thư viện cũng như bộ tăng tốc phần cứng cung cấp các tối ưu hóa dành riêng cho ReLU. Tuy nhiên, Swish có lẽ là lựa chọn mặc định tốt hơn cho các tác vụ phức tạp hơn, và bạn thậm chí có thể thử Swish tham số hóa với tham số β có thể học được cho các tác vụ phức tạp nhất. Mish có thể cho kết quả tốt hơn một chút, nhưng nó yêu cầu nhiều tài nguyên tính toán hơn. Nếu bạn quan tâm nhiều đến độ trễ thời gian chạy, thì bạn có thể thích Leaky ReLU hoặc Leaky ReLU tham số hóa cho các tác vụ phức tạp hơn. Đối với MLP sâu, hãy thử SELU, nhưng hãy đảm bảo tuân thủ các ràng buộc đã nêu ở trên. Nếu bạn có thời gian và sức mạnh tính toán dư thừa, bạn cũng có thể sử dụng phương pháp kiểm định chéo để đánh giá các hàm kích hoạt khác.

Vậy là xong phần hàm kích hoạt! Giờ, chúng ta hãy xem xét một cách hoàn toàn khác để giải quyết vấn đề độ dốc không ổn định: Chuẩn hóa theo lô (Batch Normalization).

Chuẩn hóa theo lô

Mặc dù việc sử dụng khởi tạo He cùng với hàm ReLU (hoặc bất kỳ biến thể nào của nó) có thể giảm đáng kể nguy cơ xảy ra vấn đề gradient biến mất/bùng nổ ở giai đoạn đầu huấn luyện, nhưng điều đó không đảm bảo rằng vấn đề này sẽ không quay trở lại trong quá trình huấn luyện.

Trong một [bài báo năm 2015](#)¹², Sergey Ioffe và Christian Szegedy đã đề xuất một kỹ thuật gọi là Chuẩn hóa theo lô (Batch Normalization - BN) để giải quyết những vấn đề này. Kỹ thuật này bao gồm việc thêm một phép toán vào mô hình ngay trước hoặc sau hàm kích hoạt của mỗi lớp ẩn. Phép toán này đơn giản là đưa giá trị về 0 và chuẩn hóa từng đầu vào, sau đó nhân tỷ lệ và dịch chuyển kết quả bằng cách sử dụng hai vector tham số mới cho mỗi lớp: một để nhân tỷ lệ, một để dịch chuyển.

Nói cách khác, thao tác này cho phép mô hình học được tỷ lệ và giá trị trung bình tối ưu của từng đầu vào của lớp. Trong nhiều trường hợp, nếu bạn thêm lớp BN làm lớp đầu tiên của mạng nơ-ron, bạn không cần phải chuẩn hóa tập dữ liệu huấn luyện (tức là không cần StandardScaler hoặc Normalization); lớp BN sẽ thực hiện việc đó cho bạn (gần đúng, vì nó chỉ xem xét một lô dữ liệu tại một thời điểm, và nó cũng có thể điều chỉnh tỷ lệ và dịch chuyển từng đặc trưng đầu vào).

Để chuẩn hóa và đưa các đầu vào về 0, thuật toán cần ước tính giá trị trung bình và độ lệch chuẩn của mỗi đầu vào. Thuật toán thực hiện điều này bằng cách đánh giá giá trị trung bình và độ lệch chuẩn của đầu vào trên mini-batch hiện tại (do đó có tên gọi là "Chuẩn hóa theo lô"). Toàn bộ quá trình được tóm tắt từng bước trong **Phương trình 11-4**.

Phương trình 11-4. Thuật toán chuẩn hóa theo lô

$$\begin{aligned}
 1. \quad \mu_B &= \frac{1}{m_B} \sum_{i=1}^{m_B} x^{(T)}(i) \\
 2. \quad \sigma_B^2 &= \frac{1}{m_B} \sum_{i=1}^{m_B} (x^{(T)}(i) - \mu_B)^2 \\
 3. \quad \hat{x}^{(T)}(i) &= \frac{x^{(T)}(i) - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \\
 4. \quad z(i) &= \hat{x}^{(T)}(i) + \gamma
 \end{aligned}$$

Trong thuật toán này:

- μ_B là vectơ các giá trị trung bình đầu vào, được đánh giá trên toàn bộ mini-batch B (nó chứa một giá trị trung bình cho mỗi đầu vào).
- m_B là số lượng trường hợp trong mini-batch.
- σ_B^2 là vectơ độ lệch chuẩn đầu vào, cũng được đánh giá trên toàn bộ lô nhỏ (nó chứa một độ lệch chuẩn cho mỗi đầu vào).
- $\hat{x}^{(T)}$ là vectơ các đầu vào được chuẩn hóa và có tâm bằng 0 cho ví dụ i .
- ϵ là một số rất nhỏ giúp tránh phép chia cho 0 và đảm bảo độ dốc không tăng quá lớn (thường là 10^{-5}). Đây được gọi là hệ số làm mịn.
- γ là vectơ tham số tỷ lệ đầu ra cho lớp (nó chứa một tham số tỷ lệ cho mỗi đầu vào).

- biểu thị phép nhân từng phần tử (mỗi đầu vào được nhân với tham số tỷ lệ đầu ra tương ứng).
- β là vectơ tham số dịch chuyển (độ lệch) đầu ra cho lớp (nó chứa một tham số độ lệch cho mỗi đầu vào). Mỗi đầu vào được dịch chuyển bằng tham số dịch chuyển tương ứng của nó.
- $(i) z$ là đầu ra của phép toán BN. Nó là phiên bản được điều chỉnh tỷ lệ và dịch chuyển của các đầu vào.

Vì vậy, trong quá trình huấn luyện, BN chuẩn hóa đầu vào, sau đó điều chỉnh tỷ lệ và bù trừ chúng. Tốt! Còn khi kiểm thử thì sao? Chà, mọi chuyện không đơn giản như vậy. Thực tế, chúng ta có thể cần đưa ra dự đoán cho từng trường hợp riêng lẻ chứ không phải cho cả nhóm trường hợp: trong trường hợp này, chúng ta sẽ không có cách nào để tính toán giá trị trung bình và độ lệch chuẩn của từng đầu vào. Hơn nữa, ngay cả khi chúng ta có một nhóm trường hợp, nó có thể quá nhỏ, hoặc các trường hợp có thể không độc lập và phân bố đồng nhất, do đó việc tính toán số liệu thống kê trên nhóm trường hợp sẽ không đáng tin cậy. Một giải pháp có thể là đợi đến cuối quá trình huấn luyện, sau đó chạy toàn bộ tập dữ liệu huấn luyện qua mạng nơ-ron và tính toán giá trị trung bình và độ lệch chuẩn của từng đầu vào của lớp BN. Các giá trị trung bình và độ lệch chuẩn đầu vào "cuối cùng" này sau đó có thể được sử dụng thay cho giá trị trung bình và độ lệch chuẩn đầu vào của nhóm khi đưa ra dự đoán. Tuy nhiên, hầu hết các triển khai của Batch Normalization đều ước tính các số liệu thống kê cuối cùng này trong quá trình huấn luyện bằng cách sử dụng trung bình động của giá trị trung bình và độ lệch chuẩn đầu vào của lớp. Đây là điều mà Keras tự động thực hiện khi bạn sử dụng lớp BatchNormalization. Tóm lại, bốn vectơ tham số được học trong mỗi lớp chuẩn hóa theo lô: γ (vectơ tỷ lệ đầu ra) và β (vectơ độ lệch đầu ra) được học thông qua lan truyền ngược thông thường, và μ (vectơ trung bình đầu vào cuối cùng) và σ (vectơ độ lệch chuẩn đầu vào cuối cùng) được ước tính bằng cách sử dụng trung bình động theo hàm mũ. Lưu ý rằng μ và σ được ước tính trong quá trình huấn luyện, nhưng chúng chỉ được sử dụng sau khi huấn luyện (để thay thế các giá trị trung bình và độ lệch chuẩn đầu vào theo lô trong P

Ioffe và Szegedy đã chứng minh rằng Batch Normalization cải thiện đáng kể tất cả các mạng nơ-ron sâu mà họ đã thử nghiệm, dẫn đến sự cải thiện vượt bậc trong nhiệm vụ phân loại ImageNet (ImageNet là một tập dữ liệu lớn).

(cơ sở dữ liệu hình ảnh được phân loại thành nhiều lớp, thường được sử dụng để đánh giá các hệ thống thị giác máy tính). Vấn đề suy giảm độ dốc đã được giảm thiểu đáng kể, đến mức họ có thể sử dụng các hàm kích hoạt bão hòa như tanh và thậm chí cả hàm kích hoạt sigmoid. Các mạng cũng ít nhạy cảm hơn nhiều với việc khởi tạo trọng số. Các tác giả đã có thể sử dụng tốc độ học lớn hơn nhiều, giúp tăng tốc đáng kể quá trình học. Cụ thể, họ lưu ý rằng:

Khi được áp dụng vào một mô hình phân loại hình ảnh tiên tiến, Batch Normalization đạt được độ chính xác tương tự với số bước huấn luyện ít hơn 14 lần và vượt trội hơn mô hình gốc một cách đáng kể. [...] Sử dụng một tập hợp các mạng được chuẩn hóa theo lô, chúng tôi cải thiện kết quả tốt nhất đã được công bố trên phân loại ImageNet: đạt lỗi xác thực top-5 là 4,9% (và lỗi kiểm tra là 4,8%), vượt qua độ chính xác của con người.

người đánh giá.

Cuối cùng, giống như một món quà luôn mang lại lợi ích, Batch Normalization hoạt động như một bộ điều chỉnh, giảm thiểu nhu cầu sử dụng các kỹ thuật điều chỉnh khác (chẳng hạn như dropout, được mô tả sau trong chương này).

Tuy nhiên, Batch Normalization (BN) làm tăng thêm độ phức tạp cho mô hình (mặc dù nó có thể loại bỏ nhu cầu chuẩn hóa dữ liệu đầu vào, như chúng ta đã thảo luận trước đó). Hơn nữa, nó còn gây ra tổn thất về thời gian chạy: mạng nơ-ron đưa ra dự đoán chậm hơn do các phép tính bổ sung cần thiết ở mỗi lớp. May mắn thay, thường có thể kết hợp lớp BN với lớp trước đó sau khi huấn luyện, do đó tránh được tổn thất về thời gian chạy. Điều này được thực hiện bằng cách cập nhật trọng số và độ lệch của lớp trước đó để nó trực tiếp tạo ra đầu ra có tỷ lệ và độ lệch phù hợp. Ví dụ, nếu lớp trước đó tính toán $XW + b$, thì lớp BN sẽ tính toán $\gamma (XW + b - \mu) / \sigma + \beta$ (bỏ qua số hạng làm mịn ϵ trong mẫu số). Nếu ta định nghĩa $W' = \gamma W / \sigma$ và $b' = \gamma (b - \mu) / \sigma + \beta$, phương trình sẽ đơn giản hóa thành $XW' + b'$. Vì vậy, nếu chúng ta thay thế trọng số và độ lệch (W và b) của lớp trước đó bằng trọng số và độ lệch đã cập nhật (W' và b'), chúng ta có thể loại bỏ lớp BN (bộ chuyển đổi của TFLite thực hiện việc này tự động; xem [Chương 19](#)).

GHI CHÚ

Bạn có thể thấy quá trình huấn luyện khá chậm, vì mỗi epoch mất nhiều thời gian hơn khi sử dụng Batch Normalization. Tuy nhiên, điều này thường được bù đắp bởi thực tế là quá trình hội tụ nhanh hơn nhiều với BN, do đó sẽ cần ít epoch hơn để đạt được hiệu suất tương tự. Nhìn chung, thời gian thực tế (wall time) thường sẽ ngắn hơn (thời gian này được đo bằng đồng hồ trên tường).

Triển khai chuẩn hóa hàng loạt (Batch Normalization) với Keras

Cũng như hầu hết mọi thứ với Keras, việc triển khai Batch Normalization rất đơn giản và trực quan. Chỉ cần thêm một lớp BatchNormalization trước hoặc sau hàm kích hoạt của mỗi lớp ẩn. Bạn cũng có thể thêm một lớp BN làm lớp đầu tiên trong mô hình của mình, nhưng một lớp Normalization thông thường thường hoạt động tốt ở vị trí này (nhược điểm duy nhất là bạn phải gọi phương thức `adapt()` của nó trước). Ví dụ, mô hình này áp dụng BN sau mỗi lớp ẩn và là lớp đầu tiên trong mô hình (sau khi làm phẳng ảnh đầu vào):

```
model =
    tf.keras.Sequential([ tf.keras.layers.Flatten(input_shape=[28,
28]), tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Dense(300, activation="relu",
                           kernel_initializer="he_normal"),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Dense(100, activation="relu",
                           kernel_initializer="he_normal"),
    tf.keras.layers.BatchNormalization(),
    tf.keras.layers.Dense(10, activation="softmax")
])
```

Vậy là xong! Trong ví dụ nhỏ này với chỉ hai lớp ẩn, Batch Normalization có lẽ sẽ không tạo ra tác động lớn, nhưng đối với các mạng nơ-ron sâu hơn, nó có thể tạo ra sự khác biệt rất lớn.

Hãy hiển thị bản tóm tắt mô hình:

```
>>> model.summary()
Mô hình: "tuần tự"
```

Lớp (loại)	Hình dạng đầu ra	Tham số #
làm phẳng (Flatten)	(Không có, 784)	0
batch_normalization (BatchNo (None, 784))		3136
dày đặc (Dense)	(Không có, 300)	235500
batch_normalization_1 (Batch (None, 300))		1200
dense_1 (Dense)	(Không có, 100)	30100
batch_normalization_2 (Batch (None, 100))		400
dense_2 (Dense)	(Không có, 10)	1010
=====		
Tổng số tham số: 271.346		
Số tham số có thể huấn luyện: 268.978		
Các tham số không thể huấn luyện: 2.368		

Như bạn thấy, mỗi lớp BN thêm bốn tham số cho mỗi đầu vào: γ , β , μ và

σ (ví dụ, lớp BN đầu tiên thêm 3.136 tham số, tức là 4×784).

Hai tham số cuối cùng, μ và σ , là các giá trị trung bình động; chúng không phải là

bị ảnh hưởng bởi lan truyền ngược, vì vậy Keras gọi chúng là "không thể huấn luyện"¹³ (nếu bạn

Đếm tổng số tham số BN, $3.136 + 1.200 + 400$, rồi chia

Chia cho 2, ta được 2.368, đây là tổng số tham số không thể huấn luyện được.

(trong mô hình này).

Hãy xem xét các tham số của lớp BN đầu tiên. Hai tham số có thể huấn luyện được (bởi

lan truyền ngược), và hai cái không phải là:

```
>>> [(var.name, var.trainable) for var in
model.layers[1].variables]
[('batch_normalization/gamma:0', True),
 ('batch_normalization/beta:0', True),
 ('batch_normalization/moving_mean:0', False),
 ('batch_normalization/moving_variance:0', False)]
```

Các tác giả của bài báo về BN đã lập luận ủng hộ việc thêm các lớp BN trước đó.

các hàm kích hoạt, chứ không phải sau đó (như chúng ta vừa làm). Có một số

Có một cuộc tranh luận về vấn đề này, vì phương pháp nào tốt hơn dường như phụ thuộc vào nhiệm vụ-bạn

Bạn cũng có thể thử nghiệm với điều này để xem tùy chọn nào hoạt động tốt nhất trên tập dữ liệu của mình. Để thêm các lớp BN trước các hàm kích hoạt, bạn phải loại bỏ hàm kích hoạt khỏi các lớp ẩn và thêm chúng dưới dạng các lớp riêng biệt sau các lớp BN. Hơn nữa, vì lớp Chuẩn hóa theo lô (Batch Normalization) bao gồm một tham số bù cho mỗi đầu vào, bạn có thể loại bỏ số hạng độ lệch khỏi lớp trước đó bằng cách truyền `use_bias=False` khi tạo nó. Cuối cùng, bạn thường có thể bỏ lớp BN đầu tiên để tránh việc lớp ẩn đầu tiên bị kẹt giữa hai lớp BN. Mã được cập nhật trông như thế này:

```
model =
    tf.keras.Sequential([ tf.keras.layers.Flatten(input_shape=[28,
28]),
    tf.keras.layers.Dense(300,
    kernel_initializer="he_normal",
    use_bias=False), tf.keras.layers.BatchNormalization(),

    tf.keras.layers.Activation("relu"),
    tf.keras.layers.Dense(100,
    kernel_initializer="he_normal", use_bias=False), tf.keras.layers.BatchNormaliza
])
```

Lớp BatchNormalization có khá nhiều siêu tham số mà bạn có thể điều chỉnh. Các giá trị mặc định thường sẽ ổn, nhưng đôi khi bạn có thể cần điều chỉnh tham số momentum. Siêu tham số này được lớp BatchNormalization sử dụng khi cập nhật các trung bình động theo hàm mũ; với một giá trị mới v (tức là một vectơ mới gồm các giá trị trung bình hoặc độ lệch chuẩn đầu vào được tính toán trên lô hiện tại), lớp sẽ cập nhật trung bình động $\hat{v}$ bằng phương trình sau:

$$\hat{v} = \hat{v} \times \text{động lượng} + v \times (1 - \text{động lượng})$$

Giá trị động lượng tốt thường gần bằng 1; ví dụ: 0,9, 0,99 hoặc 0,999. Bạn muốn có nhiều số 9 hơn đối với các tập dữ liệu lớn hơn và các mini-batch nhỏ hơn.

Một siêu tham số quan trọng khác là `axis`: nó xác định trục nào nên được chuẩn hóa. Giá trị mặc định là -1, có nghĩa là theo mặc định nó sẽ chuẩn hóa trục cuối cùng (sử dụng giá trị trung bình và độ lệch chuẩn đã tính toán).

(trên các trục khác). Khi batch đầu vào là 2D (tức là hình dạng batch là [kích thước batch, đặc trưng]), điều này có nghĩa là mỗi đặc trưng đầu vào sẽ được chuẩn hóa dựa trên giá trị trung bình và độ lệch chuẩn được tính toán trên tất cả các trường hợp trong batch. Ví dụ, lớp BN đầu tiên trong ví dụ mã trước đó sẽ chuẩn hóa (và điều chỉnh tỷ lệ và dịch chuyển) độc lập từng trong số 784 đặc trưng đầu vào. Nếu chúng ta di chuyển lớp BN đầu tiên lên trước lớp Flatten, thì các batch đầu vào sẽ là 3D, với hình dạng [kích thước batch, chiều cao, chiều rộng]; do đó, lớp BN sẽ tính toán 28 giá trị trung bình và 28 độ lệch chuẩn (1 cho mỗi cột pixel, được tính toán trên tất cả các trường hợp trong batch và trên tất cả các hàng trong cột), và nó sẽ chuẩn hóa tất cả các pixel trong một cột nhất định bằng cách sử dụng cùng một giá trị trung bình và độ lệch chuẩn. Cũng sẽ chỉ có 28 tham số tỷ lệ và 28 tham số dịch chuyển. Nếu thay vào đó bạn vẫn muốn xử lý từng trong số 784 pixel một cách độc lập, thì bạn nên đặt `axis=[1, 2]`.

Batch Normalization (BN) đã trở thành một trong những lớp được sử dụng nhiều nhất trong mạng nơ-ron sâu, đặc biệt là mạng nơ-ron tích chập sâu (Chương 14), đến mức nó thường bị bỏ qua trong sơ đồ kiến trúc: người ta cho rằng BN được thêm vào sau mỗi lớp. Bây giờ chúng ta hãy xem xét một kỹ thuật cuối cùng để ổn định độ dốc trong quá trình huấn luyện: Gradient Clipping (Cắt độ dốc).

Cắt xén gradient

Một kỹ thuật khác để giảm thiểu vấn đề bùng nổ độ dốc là cắt bớt độ dốc trong quá trình lan truyền ngược sao cho chúng không bao giờ vượt quá một ngưỡng nhất định.

Đây được gọi là **Cắt xén độ dốc** (Gradient ¹⁴Clipping). Kỹ thuật này thường được sử dụng trong mạng nơ-ron hồi quy, vì việc sử dụng Batch Normalization trong mạng nơ-ron hồi quy (RNN) khá phức tạp, như chúng ta sẽ thấy trong Chương 15.

Trong Keras, việc triển khai Gradient Clipping chỉ đơn giản là thiết lập tham số `clipvalue` hoặc `clipnorm` khi tạo trình tối ưu hóa, như sau:

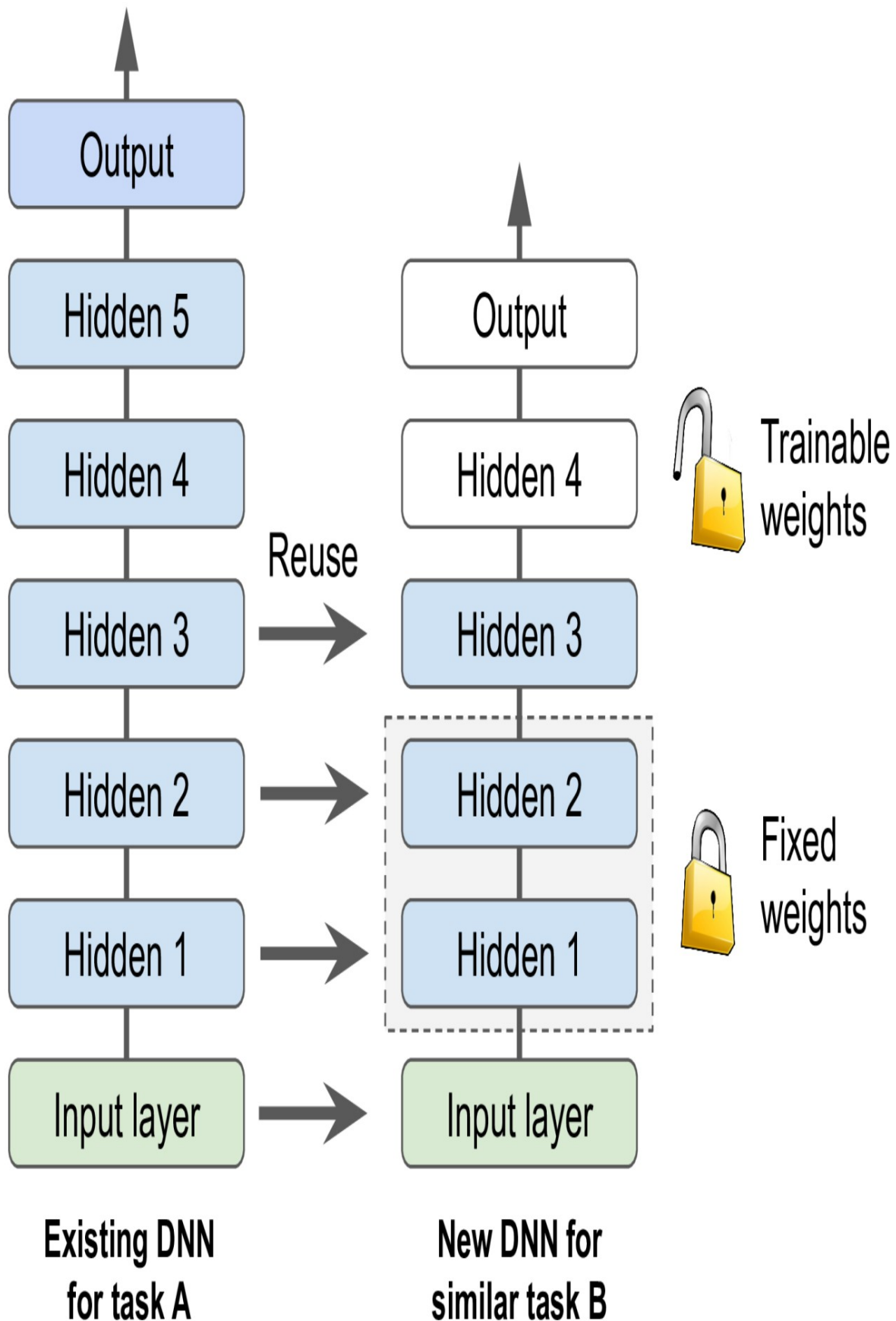
```
optimizer = tf.keras.optimizers.SGD(clipvalue=1.0)
model.compile(..., optimizer=optimizer)
```

Trình tối ưu hóa này sẽ cắt mọi thành phần của vectơ gradient thành một giá trị nằm giữa -1.0 và 1.0. Điều này có nghĩa là tất cả các đạo hàm riêng của hàm mất mát (đối với từng tham số có thể huấn luyện) sẽ được cắt trong khoảng từ -1.0 đến 1.0. Ngưỡng là một siêu tham số mà bạn có thể điều chỉnh. Lưu ý rằng nó có thể thay đổi hướng của vectơ gradient. Ví dụ, nếu vectơ gradient ban đầu là $[0.9, 100.0]$, nó chủ yếu hướng về phía trục thứ hai; nhưng khi bạn cắt nó theo giá trị, bạn sẽ nhận được $[0.9, 1.0]$, hướng gần như theo đường chéo giữa hai trục. Trên thực tế, phương pháp này hoạt động tốt. Nếu bạn muốn đảm bảo rằng Gradient Clipping không thay đổi hướng của vectơ gradient, bạn nên cắt theo chuẩn bằng cách đặt `clipnorm` thay vì `clipvalue`. Điều này sẽ cắt toàn bộ gradient nếu chuẩn l của nó lớn hơn ngưỡng bạn đã chọn. Ví dụ, nếu bạn đặt `clipnorm=1.0`, thì vectơ $[0.9, 100.0]$ sẽ bị cắt thành $[0.00899964, 0.9999595]$, giữ nguyên hướng nhưng gần như loại bỏ thành phần đầu tiên. Nếu bạn nhận thấy độ dốc tăng đột biến trong quá trình huấn luyện (bạn có thể theo dõi kích thước của độ dốc bằng TensorBoard), bạn có thể thử cắt theo giá trị hoặc cắt theo chuẩn, với các ngưỡng khác nhau, và xem tùy chọn nào hoạt động tốt nhất trên tập dữ liệu xác thực.

Tái sử dụng các lớp đã được huấn luyện trước

Thông thường, việc huấn luyện một mạng nơ-ron sâu (DNN) rất lớn từ đầu mà không tìm kiếm trước một mạng nơ-ron hiện có thực hiện nhiệm vụ tương tự là không phải là một ý tưởng hay (chúng ta sẽ thảo luận về cách tìm chúng trong [Chương 14](#)). Nếu bạn tìm thấy một mạng nơ-ron như vậy, thì bạn thường có thể tái sử dụng hầu hết các lớp của nó, ngoại trừ các lớp trên cùng. Kỹ thuật này được gọi là học chuyển giao (transfer learning). Nó không chỉ giúp tăng tốc độ huấn luyện đáng kể mà còn yêu cầu lượng dữ liệu huấn luyện ít hơn đáng kể.

Giả sử bạn có quyền truy cập vào một mạng nơ-ron sâu (DNN) đã được huấn luyện để phân loại hình ảnh thành 100 danh mục khác nhau, bao gồm động vật, thực vật, phương tiện giao thông và các vật dụng hàng ngày. Bây giờ bạn muốn huấn luyện một DNN khác để phân loại các loại phương tiện cụ thể. Các nhiệm vụ này rất giống nhau, thậm chí có phần chồng chéo, vì vậy bạn nên cố gắng tái sử dụng các phần của mạng đầu tiên (xem [Hình 11-5](#)).



Hình 11-5. Tái sử dụng các lớp đã được huấn luyện trước

GHI CHÚ

Nếu ảnh đầu vào của nhiệm vụ mới không có cùng kích thước với ảnh được sử dụng trong nhiệm vụ gốc, thông thường bạn sẽ phải thêm bước tiền xử lý để thay đổi kích thước chúng sao cho phù hợp với kích thước mà mô hình gốc mong muốn. Nói chung, học chuyển giao sẽ hoạt động tốt nhất khi các ảnh đầu vào có các đặc điểm cấp thấp tương tự nhau.

Lớp đầu ra của mô hình gốc thường cần được thay thế vì nó rất có thể không hữu ích cho nhiệm vụ mới, và nói chung nó thậm chí không có đủ số lượng đầu ra cần thiết.

Tương tự, các lớp ẩn phía trên của mô hình gốc ít có khả năng hữu ích bằng các lớp phía dưới, vì các đặc trưng cấp cao hữu ích nhất cho nhiệm vụ mới có thể khác biệt đáng kể so với các đặc trưng hữu ích nhất cho nhiệm vụ ban đầu. Bạn cần tìm số lượng lớp phù hợp để tái sử dụng.

MẸO

Các tác vụ càng giống nhau thì càng nên tái sử dụng nhiều lớp (bắt đầu từ các lớp thấp hơn). Đối với các tác vụ rất giống nhau, hãy cố gắng giữ lại tất cả các lớp ẩn và chỉ thay thế lớp đầu ra.

Hãy thử đóng băng tất cả các lớp được sử dụng lại trước (tức là làm cho trọng số của chúng không thể huấn luyện được để thuật toán Gradient Descent không sửa đổi chúng và chúng sẽ được giữ nguyên), sau đó huấn luyện mô hình của bạn và xem hiệu suất như thế nào. Sau đó, hãy thử bỏ đóng băng một hoặc hai lớp ẩn trên cùng để cho phép thuật toán lan truyền ngược tinh chỉnh chúng và xem hiệu suất có được cải thiện hay không. Bạn càng có nhiều dữ liệu huấn luyện, bạn càng có thể bỏ đóng băng nhiều lớp hơn. Việc giảm tốc độ học khi bỏ đóng băng các lớp được sử dụng lại cũng rất hữu ích: điều này sẽ tránh làm hỏng các trọng số đã được tinh chỉnh của chúng.

Nếu bạn vẫn không đạt được hiệu suất tốt và dữ liệu huấn luyện của bạn còn ít, hãy thử bỏ lớp ẩn trên cùng và đóng băng tất cả các lớp ẩn còn lại.

các lớp lại một lần nữa. Bạn có thể lặp lại cho đến khi tìm thấy số lượng lớp phù hợp để tái sử dụng. Nếu bạn có nhiều dữ liệu huấn luyện, bạn có thể thử thay thế các lớp ẩn trên cùng thay vì loại bỏ chúng, và thậm chí thêm nhiều lớp ẩn hơn.

Chuyển giao học tập với Keras

Hãy xem một ví dụ. Giả sử tập dữ liệu Fashion MNIST chỉ chứa tám lớp—ví dụ, tất cả các lớp ngoại trừ dép xăng đan và áo sơ mi. Ai đó đã xây dựng và huấn luyện một mô hình Keras trên tập dữ liệu đó và đạt được hiệu suất khá tốt (>90% độ chính xác). Chúng ta hãy gọi mô hình này là A.

Giờ bạn muốn giải quyết một nhiệm vụ khác: bạn có hình ảnh áo phông và áo len, và bạn muốn huấn luyện một bộ phân loại nhị phân: tích cực cho áo phông (và áo kiểu), tiêu cực cho dép xăng đan. Tập dữ liệu của bạn khá nhỏ; bạn chỉ có 200 hình ảnh được gắn nhãn. Khi bạn huấn luyện một mô hình mới cho nhiệm vụ này (gọi là mô hình B) với cùng kiến trúc như mô hình A, bạn nhận được độ chính xác kiểm thử là 91,85%. Trong lúc nhâm nhi cà phê buổi sáng, bạn nhận ra nhiệm vụ của mình khá giống với nhiệm vụ A, vậy có lẽ học chuyển giao có thể giúp ích? Cùng tìm hiểu nhé!

Đầu tiên, bạn cần tải mô hình A và tạo một mô hình mới dựa trên các lớp của mô hình đó. Chúng ta hãy tái sử dụng tất cả các lớp ngoại trừ lớp đầu ra:

```
[...] # Giả sử mô hình A đã được huấn luyện và lưu vào "my_model_A"
model_A =
tf.keras.models.load_model("my_model_A")
model_B_on_A =
tf.keras.Sequential(model_A.layers[:-1])
model_B_on_A.add(tf.keras.layers.Dense(1, activation="sigmoid"))
```

Lưu ý rằng model_A và model_B_on_A hiện đang chia sẻ một số lớp. Khi bạn huấn luyện model_B_on_A, nó cũng sẽ ảnh hưởng đến model_A. Nếu bạn muốn tránh điều đó, bạn cần sao chép model_A trước khi sử dụng lại các lớp của nó. Để làm điều này, bạn sao chép kiến trúc của model A bằng hàm clone_model(), sau đó sao chép trọng số của nó:

```
model_A_clone = tf.keras.models.clone_model(model_A)
model_A_clone.set_weights(model_A.get_weights())
```

CẢNH BÁO

`tf.keras.models.clone_model()` chỉ sao chép kiến trúc, chứ không sao chép trọng số. Nếu bạn không sao chép chúng thủ công bằng cách sử dụng `set_weights()`, chúng sẽ được khởi tạo ngẫu nhiên khi mô hình được sao chép được sử dụng lần đầu tiên.

Giờ bạn có thể huấn luyện `model_B_on_A` cho nhiệm vụ B, nhưng vì lớp đầu ra mới được khởi tạo ngẫu nhiên nên nó sẽ tạo ra lỗi lớn (ít nhất là trong vài epoch đầu tiên), do đó sẽ có độ dốc lỗi lớn có thể làm hỏng các trọng số được sử dụng lại. Để tránh điều này, một cách tiếp cận là đóng băng các lớp được sử dụng lại trong vài epoch đầu tiên, cho lớp mới một khoảng thời gian để học các trọng số hợp lý. Để làm điều này, hãy đặt thuộc tính `trainable` của mỗi lớp thành `False` và biên dịch mô hình:

```
for layer in model_B_on_A.layers[:-1]:
    layer.trainable = False

optimizer = tf.keras.optimizers.SGD(learning_rate=0.001)
model_B_on_A.compile(loss="binary_crossentropy",
optimizer=optimizer,
                    số liệu=["độ chính xác"])
```

GHI CHÚ

Bạn luôn phải biên dịch mô hình sau khi đóng băng hoặc bỏ đóng băng các lớp.

Giờ bạn có thể huấn luyện mô hình trong vài epoch, sau đó bỏ đóng băng các lớp được sử dụng lại (điều này yêu cầu biên dịch lại mô hình) và tiếp tục huấn luyện để tinh chỉnh các lớp được sử dụng lại cho nhiệm vụ B. Sau khi bỏ đóng băng các lớp được sử dụng lại, thường nên giảm tốc độ học, một lần nữa để tránh làm hỏng các trọng số được sử dụng lại.

```
history = model_B_on_A.fit(X_train_B, y_train_B, epochs=4,
                           validation_data=(X_valid_B,
y_valid_B))
```

```

for layer in model_B_on_A.layers[:-1]:
    layer.trainable = True

optimizer = tf.keras.optimizers.SGD(learning_rate=0.001)
model_B_on_A.compile(loss="binary_crossentropy",
optimizer=optimizer,
                    số liệu=["độ chính xác"])
history = model_B_on_A.fit(X_train_B, y_train_B, epochs=16,
                           validation_data=(X_valid_B,
y_valid_B))

```

Vậy kết luận cuối cùng là gì? Độ chính xác kiểm thử của mô hình này là 93,85%, tăng chính xác hai điểm phần trăm so với 91,85%! Điều này có nghĩa là học chuyển giao đã giảm tỷ lệ lỗi xuống gần 25%!

```

>>> model_B_on_A.evaluate(X_test_B, y_test_B)
[0.2546142041683197, 0.9384999871253967]

```

Bạn có tin không? Bạn không nên tin: Tôi đã gian lận! Tôi đã thử nhiều cấu hình cho đến khi tìm thấy một cấu hình cho thấy sự cải thiện rõ rệt. Nếu bạn thử thay đổi các lớp hoặc hạt giống ngẫu nhiên, bạn sẽ thấy rằng sự cải thiện thường giảm đi, thậm chí biến mất hoặc đảo ngược. Điều tôi đã làm được gọi là "tra tấn dữ liệu cho đến khi nó thú nhận". Khi một bài báo trông quá tích cực, bạn nên nghi ngờ: có lẽ kỹ thuật mới hào nhoáng đó thực sự không giúp ích nhiều (trên thực tế, nó thậm chí có thể làm giảm hiệu suất), nhưng các tác giả đã thử nhiều biến thể và chỉ báo cáo kết quả tốt nhất (có thể là do may mắn), mà không đề cập đến bao nhiêu thất bại họ gặp phải trên đường đi. Hầu hết thời gian, điều này hoàn toàn không có ý đồ xấu, nhưng đó là một phần lý do tại sao rất nhiều kết quả trong khoa học không bao giờ có thể được tái tạo.

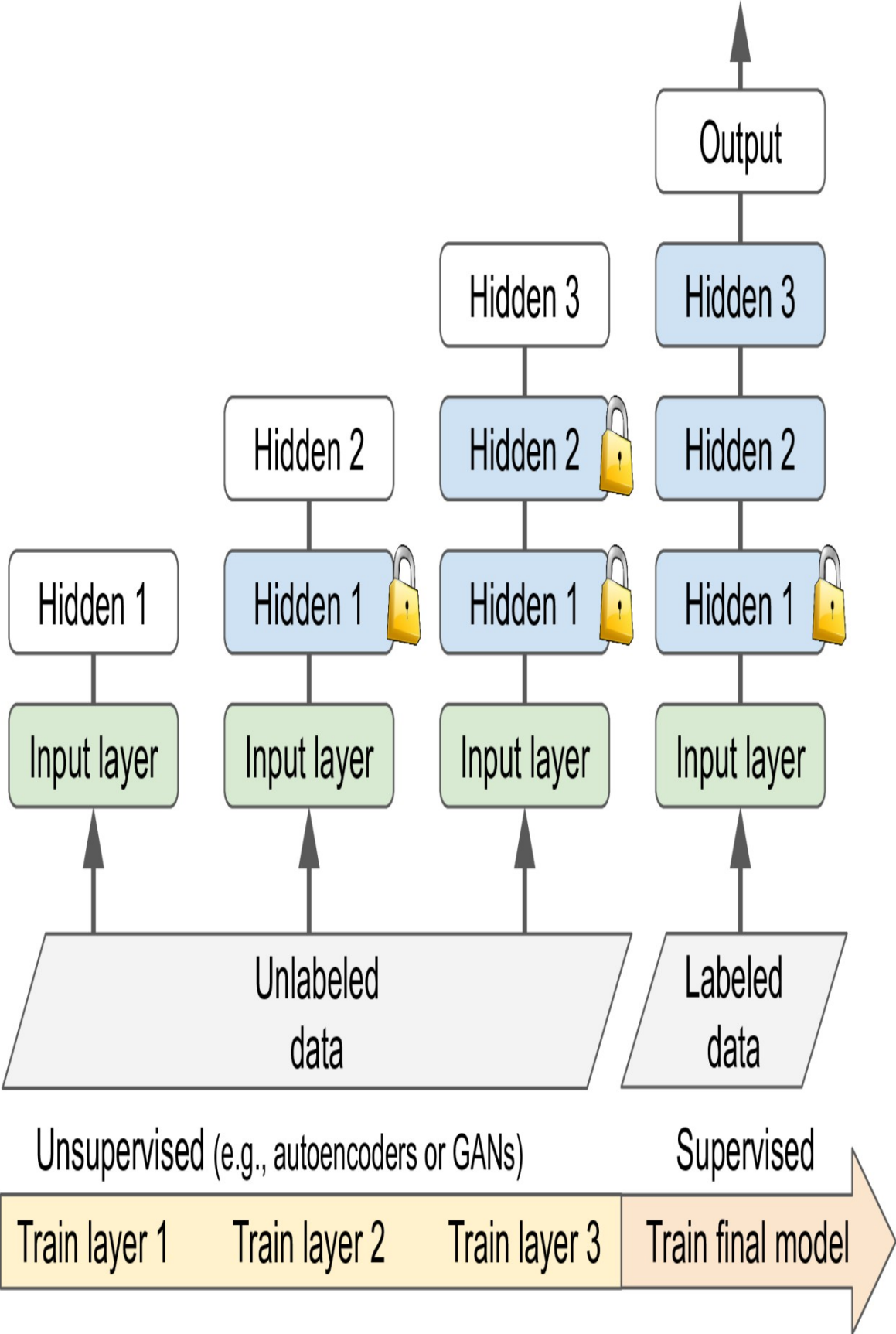
Tại sao tôi lại gian lận? Hóa ra, học chuyển giao không hoạt động hiệu quả lắm với các mạng nơ-ron nhỏ và dày đặc, có lẽ vì các mạng nhỏ học được ít mẫu, còn các mạng dày đặc học được các mẫu rất cụ thể, khó có thể hữu ích trong các nhiệm vụ khác. Học chuyển giao hoạt động tốt nhất với các mạng nơ-ron tích chập sâu, có xu hướng học các bộ phát hiện đặc trưng tổng quát hơn nhiều (đặc biệt là ở các lớp thấp hơn). Chúng ta sẽ xem xét lại học chuyển giao trong **Chương 14**, sử dụng các kỹ thuật mà chúng ta vừa thảo luận (và lần này sẽ không có gian lận nữa, tôi hứa!).

Huấn luyện trước không giám sát

Giả sử bạn muốn giải quyết một nhiệm vụ phức tạp mà bạn không có nhiều dữ liệu huấn luyện được gắn nhãn, nhưng không may là bạn không thể tìm thấy mô hình nào được huấn luyện trên một nhiệm vụ tương tự. Đừng mất hy vọng! Trước tiên, bạn nên cố gắng thu thập thêm dữ liệu huấn luyện được gắn nhãn, nhưng nếu không thể, bạn vẫn có thể thực hiện huấn luyện trước không giám sát (xem [Hình 11-6](#)). Thật vậy, việc thu thập các ví dụ huấn luyện không được gắn nhãn thường rẻ, nhưng việc gắn nhãn cho chúng lại tốn kém. Nếu bạn có thể thu thập được nhiều dữ liệu huấn luyện không được gắn nhãn, bạn có thể thử sử dụng chúng để huấn luyện một mô hình không giám sát, chẳng hạn như bộ mã hóa tự động hoặc mạng đối kháng tạo sinh (xem [Chương 17](#)). Sau đó, bạn có thể tái sử dụng các lớp thấp hơn của bộ mã hóa tự động hoặc các lớp thấp hơn của bộ phân loại của GAN, thêm lớp đầu ra cho nhiệm vụ của bạn lên trên và tinh chỉnh mạng cuối cùng bằng cách sử dụng học có giám sát (tức là với các ví dụ huấn luyện được gắn nhãn).

Chính kỹ thuật này đã được Geoffrey Hinton và nhóm của ông sử dụng vào năm 2006, dẫn đến sự hồi sinh của mạng nơ-ron và sự thành công của Học sâu (Deep Learning). Cho đến năm 2010, huấn luyện trước không giám sát—thường sử dụng máy Boltzmann hạn chế (RBM; xem sổ tay tại <https://homl.info/extra-anns>)—là tiêu chuẩn cho các mạng sâu, và chỉ sau khi vấn đề suy giảm độ dốc được giải quyết thì việc huấn luyện mạng nơ-ron sâu (DNN) hoàn toàn bằng học có giám sát mới trở nên phổ biến hơn nhiều. Huấn luyện trước không giám sát (ngày nay thường sử dụng bộ mã hóa tự động hoặc GAN thay vì RBM) vẫn là một lựa chọn tốt khi bạn có một nhiệm vụ phức tạp cần giải quyết, không có mô hình tương tự nào có thể tái sử dụng và ít dữ liệu huấn luyện được gắn nhãn nhưng lại có nhiều dữ liệu huấn luyện không được gắn nhãn.

Cần lưu ý rằng trong những ngày đầu của Học sâu, việc huấn luyện các mô hình sâu rất khó khăn, vì vậy người ta thường sử dụng kỹ thuật gọi là huấn luyện trước từng lớp tham lam (minh họa trong [Hình 11-6](#)). Đầu tiên, họ sẽ huấn luyện một mô hình không giám sát với một lớp duy nhất, thường là RBM, sau đó họ sẽ đóng băng lớp đó và thêm một lớp khác lên trên, rồi huấn luyện lại mô hình (về cơ bản chỉ là huấn luyện lớp mới), sau đó đóng băng lớp mới và thêm một lớp khác lên trên, huấn luyện lại mô hình, và cứ thế tiếp tục. Ngày nay, mọi thứ đơn giản hơn nhiều: người ta thường huấn luyện toàn bộ mô hình không giám sát trong một lần và sử dụng autoencoder hoặc GAN thay vì RBM.



Hình 11-6. Trong huấn luyện không giám sát, mô hình được huấn luyện trên tất cả dữ liệu, bao gồm cả dữ liệu chưa được gắn nhãn, bằng kỹ thuật học không giám sát, sau đó được tinh chỉnh cho nhiệm vụ cuối cùng chỉ trên dữ liệu đã được gắn nhãn bằng kỹ thuật học có giám sát; phần không giám sát có thể huấn luyện từng lớp một như được hiển thị ở đây, hoặc nó có thể huấn luyện toàn bộ mô hình trực tiếp.

Huấn luyện trước về một nhiệm vụ phụ trợ

Nếu bạn không có nhiều dữ liệu huấn luyện được gắn nhãn, một lựa chọn cuối cùng là huấn luyện mạng nơ-ron đầu tiên trên một nhiệm vụ phụ trợ mà bạn có thể dễ dàng thu thập hoặc tạo ra dữ liệu huấn luyện được gắn nhãn, sau đó tái sử dụng các lớp thấp hơn của mạng đó cho nhiệm vụ thực tế của bạn. Các lớp thấp hơn của mạng nơ-ron đầu tiên sẽ học các bộ phát hiện đặc trưng mà mạng nơ-ron thứ hai có thể tái sử dụng.

Ví dụ, nếu bạn muốn xây dựng một hệ thống nhận dạng khuôn mặt, bạn có thể chỉ có một vài bức ảnh của mỗi cá nhân—rõ ràng là không đủ để huấn luyện một bộ phân loại tốt. Việc thu thập hàng trăm bức ảnh của mỗi người sẽ không khả thi. Tuy nhiên, bạn có thể thu thập rất nhiều ảnh của những người ngẫu nhiên trên web và huấn luyện một mạng nơ-ron đầu tiên để phát hiện xem hai bức ảnh khác nhau có cùng một người hay không. Mạng như vậy sẽ học được các bộ phát hiện đặc điểm tốt cho khuôn mặt, vì vậy việc tái sử dụng các lớp thấp hơn của nó sẽ cho phép bạn huấn luyện một bộ phân loại khuôn mặt tốt chỉ với ít dữ liệu huấn luyện.

Đối với các ứng dụng xử lý ngôn ngữ tự nhiên (NLP), bạn có thể tải xuống một kho dữ liệu gồm hàng triệu tài liệu văn bản và tự động tạo dữ liệu được gắn nhãn từ đó. Ví dụ, bạn có thể ngẫu nhiên che đi một số từ và huấn luyện một mô hình để dự đoán các từ bị thiếu (ví dụ: nó nên dự đoán rằng từ bị thiếu trong câu “What ____ you saying?” có thể là “are” hoặc “were”). Nếu bạn có thể huấn luyện một mô hình đạt hiệu suất tốt trong nhiệm vụ này, thì nó sẽ đã biết khá nhiều về ngôn ngữ, và bạn chắc chắn có thể tái sử dụng nó cho nhiệm vụ thực tế của mình và tinh chỉnh nó trên dữ liệu được gắn nhãn của bạn (chúng ta sẽ thảo luận thêm về các nhiệm vụ huấn luyện trước trong [Chương 15](#)).

GHI CHÚ

Học tự giám sát là khi bạn tự động tạo nhãn từ chính dữ liệu, như trong ví dụ về che giấu văn bản, sau đó bạn huấn luyện một mô hình trên tập dữ liệu "đã được gắn nhãn" bằng các kỹ thuật học có giám sát.

Bộ tối ưu hóa nhanh hơn

Việc huấn luyện một mạng nơ-ron sâu rất lớn có thể cực kỳ chậm. Cho đến nay, chúng ta đã thấy bốn cách để tăng tốc quá trình huấn luyện (và đạt được một giải pháp tốt hơn): áp dụng chiến lược khởi tạo tốt cho trọng số kết nối, sử dụng hàm kích hoạt tốt, sử dụng chuẩn hóa theo lô (Batch Normalization), và tái sử dụng các phần của mạng đã được huấn luyện trước (có thể được xây dựng trên một nhiệm vụ phụ trợ hoặc sử dụng học không giám sát). Một cách tăng tốc đáng kể khác đến từ việc sử dụng bộ tối ưu hóa nhanh hơn bộ tối ưu hóa Gradient Descent thông thường. Trong phần này, chúng ta sẽ trình bày các bộ tối ưu hóa phổ biến nhất: tối ưu hóa động lượng (momentum optimization), Gradient gia tốc Nesterov (Nesterov Accelerated Gradient), AdaGrad, RMSProp, và cuối cùng là tối ưu hóa Adam và các biến thể của nó.

Tối ưu hóa động lượng

Hãy tưởng tượng một quả bóng bowling lăn xuống một con dốc thoải thoải trên một bề mặt nhẵn: ban đầu nó sẽ lăn chậm, nhưng sau đó sẽ nhanh chóng tăng tốc cho đến khi đạt vận tốc giới hạn (nếu có ma sát hoặc lực cản không khí). Đây là ý tưởng cốt lõi của tối ưu hóa động lượng, được Boris Polyak đề xuất vào năm 1964. Ngược lại, thuật toán Gradient Descent thông thường¹⁵ sẽ thực hiện các bước nhỏ khi dốc thoải thoải và các bước lớn khi dốc đứng, nhưng nó sẽ không bao giờ tăng tốc. Do đó, thuật toán Gradient Descent thông thường thường chậm hơn nhiều so với tối ưu hóa động lượng trong việc đạt đến điểm cực tiểu.

Hãy nhớ rằng thuật toán Gradient Descent cập nhật các trọng số θ bằng cách trực tiếp trừ đi đạo hàm của hàm chi phí $J(\theta)$ đối với các trọng số ($J'(\theta)$) nhân với tốc độ học η . Phương trình là: $\theta \leftarrow \theta - \eta J'(\theta)$. Nó không quan tâm đến các đạo hàm trước đó là gì. Nếu đạo hàm cục bộ nhỏ, thuật toán sẽ hoạt động rất chậm.

Tối ưu hóa động lượng rất quan tâm đến các đạo hàm trước đó: ở mỗi lần lặp, nó trừ đạo hàm cục bộ khỏi vectơ động lượng m (nhân với tốc độ học η), và cập nhật trọng số bằng cách cộng vectơ động lượng này (xem [Phương trình 11-5](#)). Nói cách khác, đạo hàm được sử dụng như một gia tốc, chứ không phải là tốc độ. Để mô phỏng một cơ chế ma sát nào đó và ngăn động lượng tăng quá lớn, thuật toán giới thiệu một siêu tham số mới β , được gọi là động lượng, phải được đặt trong khoảng từ 0 (ma sát cao) đến 1 (không có ma sát). Giá trị động lượng điển hình là 0,9.

Phương trình 11-5. Thuật toán động lượng

$$\begin{aligned} 1. \quad m &= \beta m - \eta J'(\theta) \\ 2. \quad \theta &= \theta + m \end{aligned}$$

Bạn có thể kiểm chứng rằng nếu độ dốc không đổi, vận tốc cuối cùng (tức là kích thước tối đa của các cập nhật trọng số) bằng độ dốc đó nhân với tốc độ học η nhân với $1 / (1 - \beta)$ (bỏ qua dấu). Ví dụ, nếu $\beta = 0,9$, thì vận tốc cuối cùng bằng 10 lần độ dốc nhân với tốc độ học, do đó tối ưu hóa động lượng sẽ nhanh hơn 10 lần so với Gradient Descent! Điều này cho phép tối ưu hóa động lượng thoát khỏi trạng thái bão hòa nhanh hơn nhiều so với Gradient Descent.

Chúng ta đã thấy trong [Chương 4](#) rằng khi các đầu vào có thang đo rất khác nhau, hàm chi phí sẽ có dạng một cái bát dài (xem [Hình 4-7](#)). Thuật toán Gradient Descent đi xuống dốc khá nhanh, nhưng sau đó lại mất rất nhiều thời gian để xuống đến đáy thung lũng. Ngược lại, thuật toán Momentum Optimization sẽ lăn xuống thung lũng ngày càng nhanh hơn cho đến khi đạt đến đáy (điểm tối ưu).

Trong các mạng nơ-ron sâu không sử dụng Batch Normalization, các lớp trên thường sẽ có đầu vào với thang đo rất khác nhau, vì vậy việc sử dụng tối ưu hóa động lượng sẽ giúp ích rất nhiều. Nó cũng có thể giúp vượt qua các điểm tối ưu cục bộ.

GHI CHÚ

Do quán tính, thuật toán tối ưu hóa có thể vượt quá điểm cần thiết một chút, sau đó quay trở lại, lại vượt quá điểm cần thiết và dao động như vậy nhiều lần trước khi ổn định ở mức tối thiểu. Đây là một trong những lý do tại sao việc có một chút ma sát trong hệ thống lại tốt: nó loại bỏ các dao động này và do đó tăng tốc độ hội tụ.

Việc triển khai tối ưu hóa động lượng trong Keras rất đơn giản: chỉ cần sử dụng trình tối ưu hóa SGD và thiết lập siêu tham số động lượng của nó, sau đó thư giãn và thu lợi nhuận!

```
optimizer = tf.keras.optimizers.SGD(learning_rate=0.001, momentum=0.9)
```

Nhược điểm duy nhất của tối ưu hóa động lượng là nó bổ sung thêm một siêu tham số cần điều chỉnh. Tuy nhiên, giá trị động lượng 0.9 thường hoạt động tốt trong thực tế và hầu như luôn nhanh hơn so với thuật toán Gradient Descent thông thường.

Phương pháp gia tốc gradient Nesterov

Một biến thể nhỏ của tối ưu hóa động lượng, được **Yurii Nesterov** đề xuất vào năm 1983, hầu như ¹⁶ luôn nhanh hơn so với tối ưu hóa động lượng thông thường.

Phương pháp Nesterov Accelerated Gradient (NAG), còn được gọi là tối ưu hóa động lượng Nesterov, đo độ dốc của hàm chi phí không phải tại vị trí cục bộ θ mà hơi phía trước theo hướng động lượng, tại $\theta + \beta m$ (xem **Phương trình 11-6**).

Phương trình 11-6. Thuật toán Gradient gia tốc Nesterov

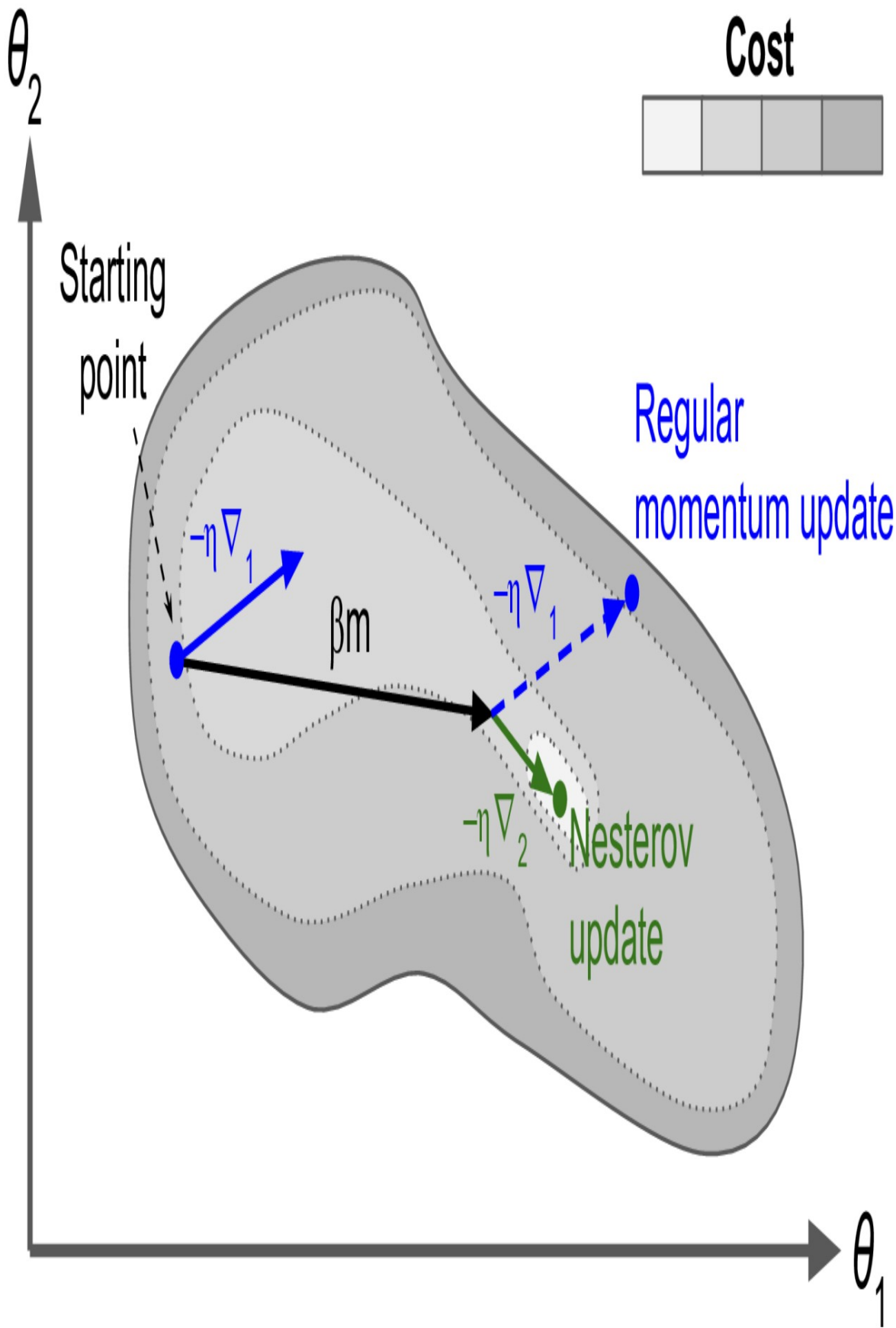
$$1. \quad m \leftarrow -\eta \nabla J(\theta + \beta m) \quad 2. \quad \theta \leftarrow \theta + m$$

Sự điều chỉnh nhỏ này có hiệu quả vì nhìn chung vectơ động lượng sẽ hướng đúng chiều (tức là hướng về điểm tối ưu), do đó việc sử dụng độ dốc được đo ở vị trí xa hơn một chút sẽ chính xác hơn.

hướng chứ không phải độ dốc tại vị trí ban đầu, như bạn có thể thấy trong **Hình 11-7** (trong đó J_1 biểu thị độ dốc của hàm chi phí được đo tại điểm bắt đầu θ , và J_2 biểu thị độ dốc tại điểm nằm ở $\theta + \beta m$).

Như bạn thấy, phương pháp cập nhật Nesterov cho kết quả gần tối ưu hơn. Sau một thời gian, những cải tiến nhỏ này cộng lại và NAG trở nên nhanh hơn đáng kể so với phương pháp tối ưu hóa động lượng thông thường. Hơn nữa, hãy lưu ý rằng khi động lượng đẩy các trọng số vượt qua một điểm lồi, tiếp tục đẩy xa hơn qua điểm lồi, trong khi đẩy ngược trở lại phía đáy điểm lồi. Điều này giúp giảm dao động và do đó NAG hội tụ nhanh hơn. NAG thường nhanh hơn so với tối ưu hóa động lượng thông thường. Để sử dụng nó, chỉ cần đặt `nesterov=True` khi tạo trình tối ưu hóa SGD:

```
optimizer = tf.keras.optimizers.SGD(learning_rate=0.001,
momentum=0.9,
nesterov=True)
```



Hình 11-7. Tối ưu hóa động lượng thông thường so với tối ưu hóa động lượng Nesterov: phương pháp trước áp dụng các đạo hàm được tính trước bước động lượng, trong khi phương pháp sau áp dụng các đạo hàm được tính sau bước động lượng.

AdaGrad

Hãy xem xét lại bài toán cái bát dài: Thuật toán Gradient Descent bắt đầu bằng cách nhanh chóng đi xuống theo con dốc đứng nhất, con dốc này không hướng thẳng đến điểm tối ưu toàn cục, sau đó nó từ từ đi xuống đáy thung lũng. Sẽ tốt hơn nếu thuật toán có thể điều chỉnh hướng đi sớm hơn để hướng đến điểm tối ưu toàn cục hơn. Thuật toán AdaGrad¹⁷ Việc hiệu chỉnh này được thực hiện bằng cách thu nhỏ vectơ độ dốc dọc theo các chiều dốc nhất (xem Phương trình 11-7).

Phương trình 11-7. Thuật toán AdaGrad

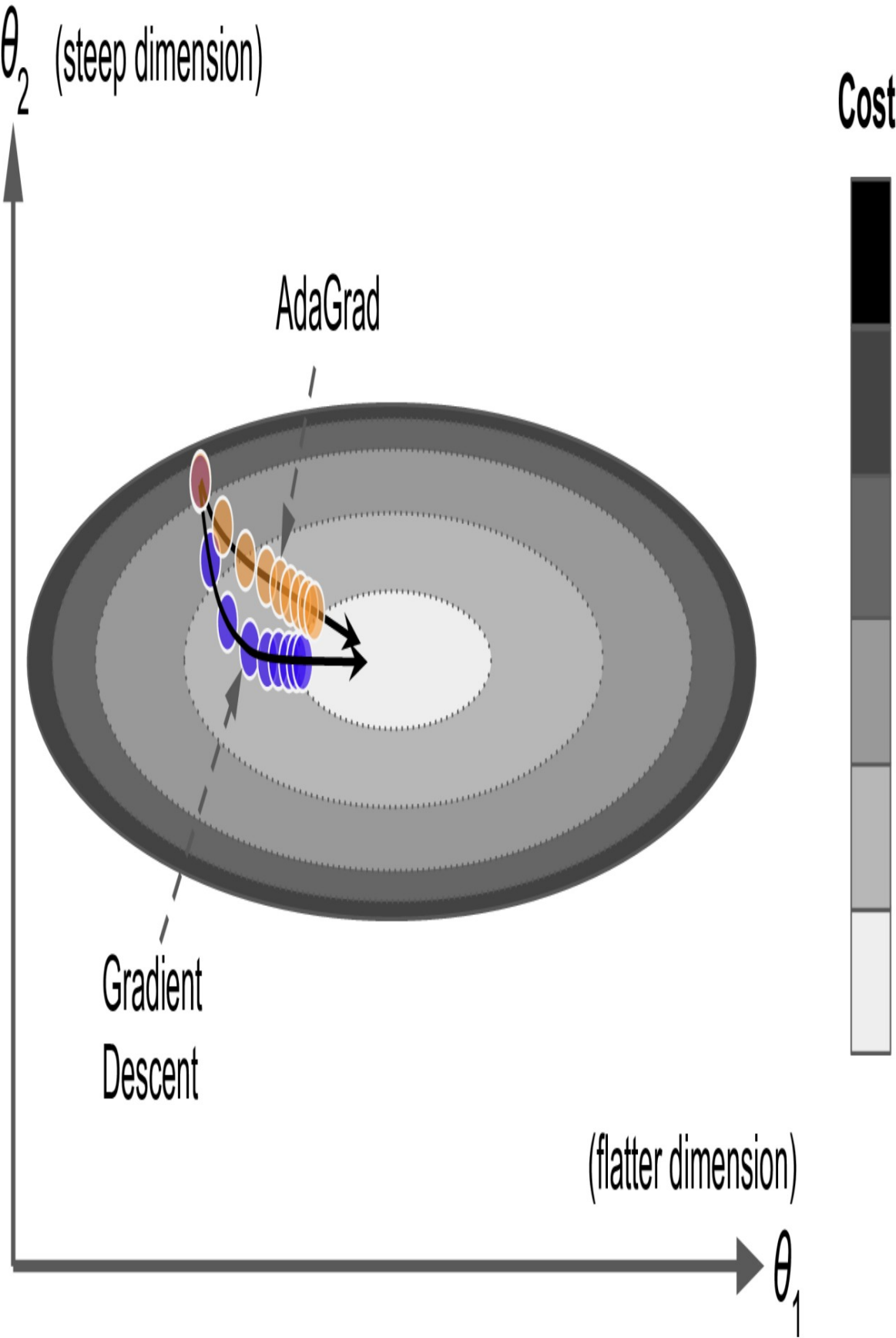
$$\begin{aligned} 1. \quad & s \leftarrow s + J(\theta) \\ 2. \quad & \eta J(\theta) \leftarrow \sqrt{s} + \epsilon \end{aligned}$$

Bước đầu tiên tích lũy bình phương của các đạo hàm riêng vào vectơ s (nhớ rằng ký hiệu $\odot$ biểu thị phép nhân từng phần tử). Dạng vectơ hóa này tương đương với việc tính $s \leftarrow s + (J(\theta) / \theta)$ cho mỗi phần tử s của vectơ s ; nói cách khác, mỗi s tích lũy bình phương của đạo hàm riêng của hàm chi phí đối với tham số θ . Nếu hàm chi phí dốc dọc theo chiều thứ i , thì s sẽ ngày càng lớn hơn ở mỗi lần lặp.

Bước thứ hai gần như giống hệt với Gradient Descent, nhưng có một điểm khác biệt lớn: vectơ gradient được thu nhỏ theo hệ số $\sqrt{s} + \epsilon$ (ký hiệu $\oslash$ biểu thị phép chia từng phần tử, và ϵ là hệ số làm mịn để tránh chia cho 0, thường được đặt là 10^{-10}). Dạng vectơ hóa này tương đương với việc tính toán đồng thời $\theta_i \leftarrow \theta_i - \eta J(\theta) / \sqrt{s_i} + \epsilon$ cho tất cả các tham số θ .

Tóm lại, thuật toán này giảm tốc độ học, nhưng giảm nhanh hơn đối với các chiều có độ dốc lớn hơn so với các chiều có độ dốc thoải thoải. Điều này được gọi là tốc độ học thích ứng. Nó giúp định hướng các bản cập nhật kết quả chính xác hơn.

hướng tới điểm tối ưu toàn cục (xem Hình 11-8). Một lợi ích bổ sung là nó yêu cầu điều chỉnh tham số siêu tham số tốc độ học ít hơn nhiều.



Hình 11-8. So sánh AdaGrad và Gradient Descent: thuật toán AdaGrad có thể điều chỉnh hướng sớm hơn để chỉ đến điểm tối ưu.

AdaGrad thường hoạt động tốt với các bài toán bậc hai đơn giản, nhưng nó thường dừng quá sớm khi huấn luyện mạng nơ-ron. Tốc độ học bị giảm xuống quá nhiều khiến thuật toán dừng hoàn toàn trước khi đạt đến điểm tối ưu toàn cục. Vì vậy, mặc dù Keras có trình tối ưu hóa AdaGrad, bạn không nên sử dụng nó để huấn luyện mạng nơ-ron sâu (mặc dù nó có thể hiệu quả đối với các tác vụ đơn giản hơn như hồi quy tuyến tính). Tuy nhiên, hiểu về AdaGrad rất hữu ích để nắm bắt các trình tối ưu hóa tốc độ học thích ứng khác.

RMSPProp

Như chúng ta đã thấy, AdaGrad có nguy cơ chậm lại quá nhanh và không bao giờ hội tụ đến điểm tối ưu toàn cục. Thuật toán RMSPProp khắc phục điều này bằng cách chỉ tích lũy các ¹⁸ gradient từ các lần lặp gần đây nhất, thay vì tất cả các gradient kể từ khi bắt đầu huấn luyện. Nó thực hiện điều này bằng cách sử dụng sự suy giảm theo hàm mũ trong bước đầu tiên (xem [Phương trình 11-8](#)).

Phương trình 11-8. Thuật toán RMSPProp

$$\begin{aligned} 1. \quad s &= \rho s + (1 - \rho) J'(\theta)^2 \\ 2. \quad \eta J'(\theta) &= \frac{\eta J'(\theta)}{\sqrt{s} + \epsilon} \end{aligned}$$

Tốc độ phân rã ρ thường được đặt là 0,9. Đúng vậy, đây lại là một siêu tham số mới, nhưng giá trị mặc định này thường hoạt động tốt, vì vậy bạn có thể không cần phải điều chỉnh nó.

Như bạn có thể đoán được, Keras có trình tối ưu hóa RMSprop:

```
optimizer = tf.keras.optimizers.RMSprop(learning_rate=0.001, rho=0.9)
```

Ngoại trừ những bài toán rất đơn giản, thuật toán tối ưu này hầu như luôn hoạt động tốt hơn nhiều so với AdaGrad. Trên thực tế, nó từng là thuật toán tối ưu hóa được nhiều nhà nghiên cứu ưa chuộng cho đến khi thuật toán tối ưu Adam ra đời.

Tối ưu hóa Adam

Adam, Thuật toán này, viết tắt của ước lượng mômen thích ứng, kết hợp các ý tưởng của tối ưu hóa động lượng và RMSProp: giống như tối ưu hóa động lượng, nó theo dõi giá trị trung bình giảm dần theo hàm mũ của các độ dốc trong quá khứ; và giống như RMSProp, nó theo dõi giá trị trung bình giảm dần theo hàm mũ của các bình phương độ dốc trong quá khứ (xem [Phương trình 11-9](#)). Đây là các ước lượng về giá trị trung bình và phương sai (không được căn giữa) của các độ dốc. Giá trị trung bình thường được gọi là mômen bậc nhất trong khi phương sai thường được gọi là mômen bậc hai, do đó có tên gọi là thuật toán.

Phương trình 11-9. Thuật toán Adam

$$\begin{aligned}
 1. \quad m &= \beta_1 m + (1 - \beta_1) J'(\theta) \\
 2. \quad s &= \beta_2 s + (1 - \beta_2) J'(\theta)^2 \\
 3. \quad \hat{m} &= \frac{m}{1 - \beta_1^t} \\
 4. \quad \hat{s} &= \frac{s}{1 - \beta_2^t} \\
 5. \quad \theta &= \theta - \eta \frac{\hat{m}}{\sqrt{\hat{s}}} + \epsilon
 \end{aligned}$$

Trong phương trình này, t biểu thị số lần lặp (bắt đầu từ 1).

Nếu bạn chỉ nhìn vào các bước 1, 2 và 5, bạn sẽ nhận thấy sự tương đồng chặt chẽ của Adam với cả tối ưu hóa động lượng và RMSProp: β tương ứng với β trong tối ưu hóa động lượng, và β tương ứng với β trong RMSProp.

Điểm khác biệt duy nhất là bước 1 tính toán giá trị trung bình giảm dần theo hàm mũ thay vì tổng giảm dần theo hàm mũ, nhưng thực tế chúng tương đương nhau ngoại trừ một hằng số (giá trị trung bình giảm dần chỉ bằng $1 - \beta$ lần tổng giảm dần). Bước 3 và 4 là một số chi tiết kỹ thuật: vì m và s được khởi tạo bằng 0, chúng sẽ có xu hướng nghiêng về 0 khi bắt đầu huấn luyện, vì vậy hai bước này sẽ giúp tăng m và s khi bắt đầu huấn luyện.

Thông số siêu tham số suy giảm động lượng β thường được khởi tạo là 0,9, trong khi thông số siêu tham số suy giảm tỷ lệ β thường được khởi tạo là 0,999. Như vậy

Trước đây, hệ số làm mịn ϵ thường được khởi tạo bằng một số rất nhỏ, chẳng hạn như 10^{-7} . Đây là các giá trị mặc định cho lớp Adam. Sau đây là cách tạo

trình tối ưu hóa Adam sử dụng Keras:

```
optimizer = tf.keras.optimizers.Adam(learning_rate=0.001, beta_1=0.9,
                                      beta_2=0.999)
```

Vì Adam là một thuật toán có tốc độ học thích ứng, giống như AdaGrad và RMSProp, nên nó yêu cầu ít điều chỉnh tham số siêu tham số tốc độ học η hơn. Bạn thường có thể sử dụng giá trị mặc định $\eta = 0.001$, làm cho Adam thậm chí còn dễ sử dụng hơn cả Gradient Descent.

MẸO

Nếu bạn bắt đầu cảm thấy choáng ngợp trước tất cả các kỹ thuật khác nhau này và đang băn khoăn không biết chọn kỹ thuật nào phù hợp với nhiệm vụ của mình, đừng lo lắng: một số hướng dẫn thực tế được cung cấp ở cuối chương này.

Cuối cùng, có ba biến thể của Adam đáng được nhắc đến: AdaMax, Nadam và AdamW.

AdaMax

Bài báo của Adam cũng giới thiệu AdaMax. Lưu ý rằng ở bước 2 của **Phương trình 11-9**, Adam tích lũy bình phương của các gradient trong s (với trọng số lớn hơn cho các gradient gần đây hơn). Ở bước 5, nếu chúng ta bỏ qua ϵ và các bước 3 và 4 (dù sao thì đó cũng chỉ là chi tiết kỹ thuật), Adam sẽ giảm tỷ lệ cập nhật tham số bằng căn bậc hai của s . Nói tóm lại, Adam giảm tỷ lệ cập nhật tham số bằng chuẩn ℓ_2 của các gradient suy giảm theo thời gian (nhớ rằng chuẩn ℓ_2 là căn bậc hai của tổng bình phương).

AdaMax thay thế chuẩn ℓ_2 bằng chuẩn ℓ_∞ (một cách nói hoa mỹ hơn để chỉ giá trị lớn nhất). Cụ thể, nó thay thế bước 2 trong **Phương trình 11-9** bằng

$s = \max(\beta_2 s, \text{abs}(J(\theta)))$, Nó bỏ qua bước 4, và ở bước 5 nó điều chỉnh tỷ lệ.

Độ dốc giảm dần theo hệ số s , là giá trị lớn nhất của giá trị tuyệt đối của độ dốc giảm theo thời gian.

Trên thực tế, điều này có thể làm cho AdaMax ổn định hơn Adam, nhưng thực sự phụ thuộc vào tập dữ liệu, và nói chung Adam hoạt động tốt hơn. Vì vậy, đây chỉ là một trình tối ưu hóa khác mà bạn có thể thử nếu gặp sự cố với Adam trên một số tác vụ.

Nadam

Tối ưu hóa Nadam là sự kết hợp giữa tối ưu hóa Adam và thủ thuật Nesterov, do đó nó thường hội tụ nhanh hơn một chút so với Adam. Trong [báo cáo giới thiệu kỹ thuật này](#), nhà nghiên cứu ²¹ Timothy Dozat đã so sánh nhiều thuật toán tối ưu khác nhau trên nhiều nhiệm vụ và nhận thấy rằng Nadam nhìn chung hoạt động tốt hơn Adam nhưng đôi khi lại bị RMSProp vượt trội hơn.

AdamW

²² **AdamW** AdamW là một biến thể của Adam tích hợp kỹ thuật điều chỉnh gọi là giảm trọng số (weight decay). Giảm trọng số làm giảm kích thước trọng số của mô hình ở mỗi lần lặp huấn luyện bằng cách nhân chúng với một hệ số giảm như 0,99. Điều này có thể gợi nhớ đến điều chỉnh ℓ (được giới thiệu trong [Chương 4](#)), cũng nhằm mục đích giữ cho trọng số nhỏ, và thực tế có thể chứng minh bằng toán học rằng điều chỉnh ℓ tương đương với giảm trọng số khi sử dụng SGD. Tuy nhiên, khi sử dụng Adam hoặc các biến thể của nó, điều chỉnh ℓ và giảm trọng số không tương đương: trên thực tế, việc kết hợp Adam với điều chỉnh ℓ dẫn đến các mô hình thường không tổng quát hóa tốt như các mô hình được tạo ra bởi SGD. AdamW khắc phục vấn đề này bằng cách kết hợp Adam với giảm trọng số một cách thích hợp. 2

CẢNH BÁO

Các phương pháp tối ưu hóa thích ứng (bao gồm RMSProp, Adam, AdaMax, Nadam và AdamW) thường rất tốt, hội tụ nhanh đến một giải pháp tối ưu. Tuy nhiên, một ²³ bài báo năm 2017 Nghiên cứu của Ashia C. Wilson và cộng sự đã chỉ ra rằng chúng có thể dẫn đến các giải pháp có khả năng khái quát hóa kém trên một số tập dữ liệu. Vì vậy, khi bạn thất vọng về hiệu suất của mô hình, hãy thử sử dụng thuật toán Nesterov Accelerated Gradient thay thế: tập dữ liệu của bạn có thể không phù hợp với thuật toán gradient thích ứng. Ngoài ra, hãy xem các nghiên cứu mới nhất, vì nó đang phát triển rất nhanh.

Để sử dụng Nadam hoặc AdaMax trong Keras, chỉ cần thay thế `tf.keras.optimizers.Adam` bằng `tf.keras.optimizers.Nadam` hoặc `tf.keras.optimizers.Adamax`. AdamW hiện chưa được tích hợp trong Keras (tính đến TF 2.9.0), nhưng nó đang được bổ sung nên có thể sẽ có sẵn vào thời điểm bạn đọc bài viết này (vui lòng kiểm tra tài liệu). Trong thời gian chờ đợi, AdamW có sẵn trong thư viện TensorFlow Add-Ons. Nếu bạn đã làm theo hướng dẫn cài đặt tại <https://homl.info/install> Để chạy mọi thứ cục bộ, bạn đã cài đặt sẵn thư viện này, nhưng nếu bạn đang sử dụng Google Colab, thì bạn cần chạy lệnh `%pip install -q -U tensorflow-addons`. Sau đó, bạn có thể sử dụng trình tối ưu hóa `tfa.optimizers.AdamW` giống như trình tối ưu hóa `tf.keras.optimizers.Adam`, với tham số siêu tham số `weight_decay` bổ sung (cần được tinh chỉnh):

```
import tensorflow_addons as tfa

optimizer = tfa.optimizers.AdamW(weight_decay=1e-5,
                                  learning_rate=0.001,
                                  beta_1=0.9, beta_2=0.999)
```

Tất cả các kỹ thuật tối ưu hóa đã thảo luận cho đến nay chỉ dựa vào các đạo hàm riêng bậc nhất (ma trận Jacobian). Tài liệu về tối ưu hóa cũng chứa các thuật toán tuyệt vời dựa trên các đạo hàm riêng bậc hai (ma trận Hessian, là các đạo hàm riêng của ma trận Jacobian). Tuy nhiên, thật không may, các thuật toán này rất khó áp dụng cho mạng nơ-ron sâu vì

Có n ma trận Hessian cho mỗi đầu ra (trong đó n là số tham số), trái ngược với chỉ n ma trận Jacobian cho mỗi đầu ra. Vì mạng nơ-ron sâu (DNN) thường có hàng chục nghìn tham số trở lên, các thuật toán tối ưu hóa bậc hai thường thậm chí không vừa với bộ nhớ, và ngay cả khi chúng vừa, việc tính toán các ma trận Hessian cũng quá chậm.

HUẤN LUYỆN MÔ HÌNH THỪA

Tất cả các thuật toán tối ưu hóa mà chúng ta vừa thảo luận đều tạo ra các mô hình dày đặc, có nghĩa là hầu hết các tham số sẽ khác không. Nếu bạn cần một mô hình cực nhanh khi chạy hoặc nếu bạn cần nó chiếm ít bộ nhớ hơn, bạn có thể thích sử dụng mô hình thưa hơn.

Một cách để đạt được điều này là huấn luyện mô hình như bình thường, sau đó loại bỏ các trọng số nhỏ (đặt chúng bằng không). Tuy nhiên, điều này thường không dẫn đến một mô hình rất thưa thớt, và nó có thể làm giảm hiệu suất của mô hình.

Một lựa chọn tốt hơn là áp dụng chuẩn hóa l_1 mạnh trong quá trình huấn luyện (chúng ta sẽ xem cách thực hiện sau trong chương này), vì nó thúc đẩy trình tối ưu hóa loại bỏ càng nhiều trọng số càng tốt (như đã thảo luận trong phần “[Hồi quy Lasso](#)” ở [Chương 4](#)).

Nếu các kỹ thuật này vẫn chưa đủ, hãy tham khảo [TensorFlow Model Optimization Toolkit \(TF-MOT\)](#), công cụ này cung cấp API cắt tỉa có khả năng loại bỏ các kết nối một cách lặp đi lặp lại trong quá trình huấn luyện dựa trên độ lớn của chúng.

Bảng 11-2 so sánh tất cả các thuật toán tối ưu hóa mà chúng ta đã thảo luận cho đến nay (* là kém, ** là trung bình và *** là tốt).

T

Một

b

le

1

1

-

2

.

0

p

ti

m

iz

e

r

c

o

m

P

Một

ri

S

o

N

Lớp học	Tốc độ hội tụ	Chất lượng hội tụ
---------	---------------	-------------------

SGD	*	***
-----	---	-----

SGD (động lượng =...)	**	***
-----------------------	----	-----

SGD(động lượng=..., Nesterov=True) **		***
---------------------------------------	--	-----

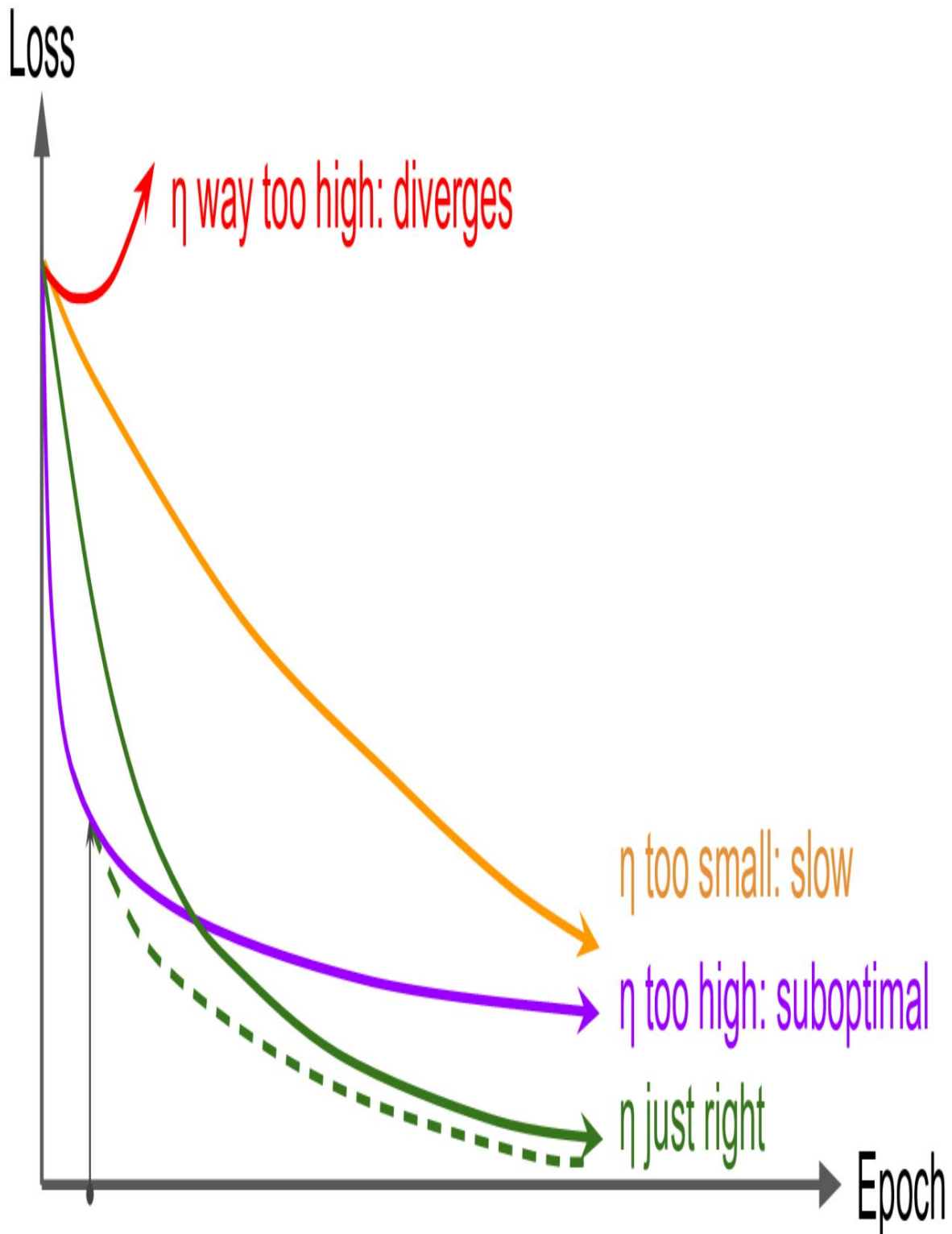
Adagrad	***	* (dừng quá sớm)
RMSprop	***	** hoặc ***
Adam	***	** *** hoặc
AdaMax	***	** *** hoặc
Nadam	***	** *** hoặc
AdamW	***	** *** hoặc

Lập lịch tốc độ học tập

Việc tìm ra tốc độ học phù hợp rất quan trọng. Nếu bạn đặt tốc độ học quá cao, Quá trình huấn luyện có thể phân kỳ (như chúng ta đã thảo luận trong “Phương pháp giảm độ dốc”). Nếu bạn thiết lập nó Nếu nhiệt độ quá thấp, quá trình huấn luyện cuối cùng sẽ hội tụ về mức tối ưu, nhưng sẽ mất một khoảng thời gian. rất lâu. Nếu bạn đặt hơi cao hơn một chút, tiến độ sẽ rất chậm.

Ban đầu thì nhanh, nhưng cuối cùng nó sẽ dao động quanh mức tối ưu, không bao giờ thực sự đạt được mức tối ưu. ổn định cuộc sống. Nếu bạn có ngân sách máy tính hạn chế, bạn có thể phải làm gián đoạn quá trình huấn luyện trước khi nó hội tụ đúng cách, dẫn đến kết quả không tối ưu.

giải pháp (xem Hình 11-9).



Start with a high learning rate then reduce it: perfect!

Hình 11-9. Đường cong học tập cho các tốc độ học tập khác nhau η

Như chúng ta đã thảo luận trong **Chương 10**, bạn có thể tìm được tốc độ học phù hợp bằng cách huấn luyện mô hình trong vài trăm lần lặp, tăng dần tốc độ học theo cấp số nhân từ một giá trị rất nhỏ đến một giá trị rất lớn, sau đó xem xét đường cong học tập và chọn một tốc độ học thấp hơn một chút so với tốc độ mà tại đó đường cong học tập bắt đầu tăng vọt trở lại. Sau đó, bạn có thể khởi tạo lại mô hình và huấn luyện nó với tốc độ học đó.

Nhưng bạn có thể làm tốt hơn là chỉ sử dụng tốc độ học không đổi: nếu bạn bắt đầu với tốc độ học lớn và sau đó giảm dần khi quá trình huấn luyện không còn tiến triển nhanh nữa, bạn có thể đạt được giải pháp tốt nhanh hơn so với tốc độ học không đổi tối ưu. Có nhiều chiến lược khác nhau để giảm tốc độ học trong quá trình huấn luyện. Bắt đầu với tốc độ học thấp, tăng dần rồi lại giảm xuống cũng có thể mang lại lợi ích. Các chiến lược này được gọi là lịch trình học (chúng ta đã giới thiệu ngắn gọn khái niệm này trong **Chương 4**). Đây là những lịch trình học được sử dụng phổ biến nhất:

Lập lịch trình năng lượng

Đặt tốc độ học là một hàm của số lần lặp t : $\eta(t) = \eta / (1 + t/s)$. 0

^c Tốc độ học ban đầu η , lũy thừa c (thường được đặt là 1) và số bước s là các siêu tham số. Tốc độ học giảm dần ở mỗi bước.

Sau s bước, nó giảm xuống còn $\eta/2$. Sau s bước nữa, nó giảm xuống còn $\eta/4$, sau đó giảm xuống $\eta/8$, rồi $\eta/16$, và cứ thế tiếp tục. Như bạn thấy, điều này làm cho lịch trình ban đầu giảm nhanh, sau đó giảm dần chậm hơn. Tất nhiên, việc lập lịch công suất đòi hỏi phải điều chỉnh η và s (và có thể cả c).

Lập lịch theo cấp số mũ

Đặt tốc độ học là $\eta(t) = \eta \cdot 0.1^{t/s}$. 0 Tốc độ học sẽ giảm dần đi 10 lần sau mỗi s bước. Trong khi lập lịch theo công suất làm giảm tốc độ học ngày càng chậm hơn, lập lịch theo hàm mũ lại tiếp tục giảm tốc độ học đi 10 lần sau mỗi s bước.

Lập lịch trình từng phần không đổi

Sử dụng tốc độ học không đổi trong một số epoch (ví dụ: $\eta = 0.1$ trong 5 epoch), sau đó sử dụng tốc độ học nhỏ hơn trong một số epoch khác (ví dụ:

$\eta = 0,001$ trong 50 epoch), và cứ thế tiếp tục. Mặc dù giải pháp này có thể hoạt động rất tốt, nhưng nó đòi hỏi phải thử nghiệm nhiều lần để tìm ra trình tự tốc độ học phù hợp và thời gian sử dụng mỗi tốc độ.

Lập kế hoạch biểu diễn

Đo lỗi xác thực sau mỗi N bước (tương tự như khi dừng sớm), và giảm tốc độ học xuống một hệ số λ khi lỗi ngừng giảm.

Lập lịch chu kỳ 1

Chu trình 1 được giới thiệu trong một [bài báo năm 2018](#)²⁴. Phương pháp này, do Leslie Smith đề xuất, trái ngược với các phương pháp khác, bắt đầu bằng cách tăng tốc độ học ban đầu η , tăng tuyến tính lên đến η ở giữa quá trình huấn luyện. Sau đó, nó giảm tốc độ học tuyến tính xuống lại η trong nửa sau của quá trình huấn luyện, và kết thúc vài epoch cuối cùng bằng cách giảm tốc độ xuống vài bậc độ lớn (vẫn theo tuyến tính). Tốc độ học tối đa η được chọn bằng cách sử dụng cùng một phương pháp mà chúng ta đã sử dụng để tìm tốc độ học tối ưu, và tốc độ học ban đầu η thường thấp hơn 10 lần. Khi sử dụng động lượng, chúng ta bắt đầu với động lượng cao trước (ví dụ: 0,95), sau đó giảm dần xuống động lượng thấp hơn trong nửa đầu quá trình huấn luyện (ví dụ: xuống 0,85, theo đường thẳng), và sau đó đưa nó trở lại giá trị tối đa (ví dụ: 0,95) trong nửa sau của quá trình huấn luyện, kết thúc vài epoch cuối cùng với giá trị tối đa đó. Smith đã thực hiện nhiều thí nghiệm cho thấy cách tiếp cận này thường có thể tăng tốc độ huấn luyện đáng kể và đạt được hiệu suất tốt hơn. Ví dụ, trên tập dữ liệu hình ảnh CIFAR10 phổ biến, cách tiếp cận này đã đạt được độ chính xác xác thực 91,9% chỉ trong 100 epoch, thay vì độ chính xác 90,3% trong 800 epoch thông qua cách tiếp cận tiêu chuẩn (với cùng kiến trúc mạng nơ-ron). Thành tựu này được gọi là siêu hội tụ.

Một [bài báo năm 2013](#)²⁵ Andrew Senior và cộng sự đã so sánh hiệu suất của một số lịch trình học tập phổ biến nhất khi sử dụng tối ưu hóa động lượng để huấn luyện mạng nơ-ron sâu cho nhận dạng giọng nói. Các tác giả kết luận rằng, trong thiết lập này, cả lập lịch hiệu suất và tối ưu hóa theo cấp số mũ đều không hiệu quả.